

Database Management System — ISRO CSE Revision Notes

Concise notes + relevant exam questions, compressed from the KnowledgeGate DBMS module.

1. DBMS Basics, Architecture & Languages

1.1 Data, Information & Database

- **Data** = raw facts/figures (qualitative or quantitative). **Information** = processed data placed in context. **Database** = an organized, electronically stored/accessed collection of data.
- **DBMS** = software that lets users/applications define, create, maintain, and control access to a database — solving the problems of raw "flat file" systems.

1.2 Problems with Traditional File Systems (why DBMS exists)

- **Data redundancy & inconsistency** — same fact stored in multiple files, updated in one but not another.
- **Difficulty in accessing data** — no query capability; needs a new program for every new access pattern.
- **Data isolation** — data scattered across incompatible file formats, hard to combine.
- **Integrity problems** — no built-in way to enforce rules (e.g., marks must be 0–100).
- **Atomicity problems** — a crash mid-operation leaves data in a half-updated (inconsistent) state.
- **Concurrent-access anomalies** — simultaneous edits by multiple users overwrite each other.
- **Security problems** — no fine-grained access control per user/role.
- **TRAP:** These 6 problems (redundancy, access difficulty, isolation, integrity, atomicity, concurrency + security) are a classic "list all problems of file-based systems" recall question — don't merge atomicity and concurrency, they are distinct failure modes (crash-related vs simultaneous-access-related).

1.3 Three-Schema Architecture (View of Data)

Level	Describes	Who works here
Physical/Internal	How data is actually stored on disk (lowest level)	DBA only
Logical/Conceptual	What data is stored + relationships (structure, entities, types)	Database designers
View/External	What a specific user sees (subset relevant to them)	End users

- **Data Independence:**
- **Logical Data Independence** — change the conceptual schema without changing external schemas/application programs.
- **Physical Data Independence** — change the internal schema (e.g., add an index, reorganize files) without changing the conceptual schema.
- **TRAP:** Physical data independence is *easier* to achieve than logical data independence, since application programs are typically tied more closely to the conceptual/logical structure than to physical storage details.

1.4 Schema vs Instance

- **Schema** = the database's fixed design/structure (like a variable's declared type) — defined once, rarely changes.
- **Instance** = the actual data present at a given moment (like a variable's current value) — changes constantly.

1.5 DBA Functions

Schema definition, storage & access-method design, schema/physical modification, granting authorization, routine maintenance (backups, monitoring disk space/performance).

1.6 OLTP vs OLAP

Feature	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Goal	Day-to-day operations	Planning/decision-making
DB size	MB–GB	TB–PB
Data source	Live/current transactions	Historical, consolidated (Data Warehouse)
Query	Short, simple, fast	Complex, long-running
Updates	Continuous (insert/delete)	Periodic batch refresh
Design	Normalized (many small tables)	De-normalized (fewer, larger tables)
Example	Booking a flight, bank balance check	5-year sales trend analysis

1.7 Data Models (Types of Databases)

- **Hierarchical:** tree structure, each child has exactly one parent — rigid.
- **Network:** child can have multiple parents — graph-like, more flexible than hierarchical.
- **Relational:** data in tables (rows=tuples/records, columns=attributes) — most widely used; every value has a unique identity via its record.
- **Object-Oriented:** data stored as objects (attributes + methods) — suited to complex data (multimedia, large files).
- **TRAP:** Network model **generalizes** the hierarchical model by allowing multiple parents per child — don't describe them as unrelated models.

1.8 DBMS Languages

DDL (Data Definition Language)	-> CREATE, ALTER, DROP, TRUNCATE, RENAME	[defines schema]
DML (Data Manipulation Language)	-> SELECT, INSERT, UPDATE, DELETE	[manipulates data]
DCL (Data Control Language)	-> GRANT, REVOKE	[access control/permissions]
TCL (Transaction Control Lang.)	-> COMMIT, ROLLBACK, SAVEPOINT	[manages transactions]

- **DML sub-types:** High-level/non-procedural (SQL — states *what* data, not *how*) vs Low-level/procedural (must be embedded in a host language like C/Java, states *how* to get it record-by-record).
- **DBMS Interfaces:** stand-alone query interface (e.g., SQL*Plus), programmer interface (embedded SQL, SQLJ), procedure-call APIs (JDBC for Java, ODBC for others), database programming languages (PL/SQL), scripting (PHP/Python/JS for web apps), user-friendly menu/forms/graphics-based interfaces, mobile interfaces.
- **TRAP:** DDL statements (like CREATE TABLE) cause an **implicit commit** in most RDBMSs — they cannot be rolled back like DML statements.

1.9 Centralized vs Client-Server Architecture

- **Centralized DBMS:** DBMS software, hardware, application programs, and UI processing all combined in a single system; remote terminals may connect, but ALL processing happens at the central site.
- **Client-Server (2-tier):** Client (application/UI) directly talks to the Database server; simple, but business logic lives on the client.
- **Client-Server (3-tier):** Client → **Application/Business-logic server** → Database server. Separates presentation, logic, and data layers — better scalability, security, and maintainability; standard for web applications.
- **TRAP:** In 2-tier architecture, the application logic resides on the *client* side; in 3-tier, it's moved to a *middle application-server tier* — this shift is the key exam distinction.

Questions

Q1. Which of the following is NOT typically listed as a problem of traditional file-based systems?

A. Data redundancy and inconsistency B. Difficulty in accessing data C. Built-in concurrency control D. Data isolation

Answer: C. **Why:** Traditional file systems *lack* built-in concurrency control — this is exactly the problem DBMS solves, not a feature file systems have.

Q2. Adding an index to speed up search, without altering the conceptual schema or any application program, is an example of:

A. Logical data independence B. Physical data independence C. View-level modification D. Schema-instance separation

Answer: B. **Why:** Changing internal storage structures (like adding an index) while the conceptual schema stays fixed is precisely physical data independence.

Q3. [Trap] The "instance" of a database refers to:

A. The fixed structural design of the database B. The actual data stored at a specific point in time C. The DBA's access rights D. The query language used

Answer: B. **Why:** Schema = fixed structure/design; Instance = the actual data content, which changes over time while the schema stays constant.

Q4. Which DBMS language category includes GRANT and REVOKE?

A. DDL B. DML C. DCL D. TCL

Answer: C. **Why:** DCL (Data Control Language) manages permissions/access control via GRANT and REVOKE.

Q5. OLAP systems are best characterized by:

A. Small database size, simple fast queries, continuous updates B. Large historical data, complex long-running queries, periodic batch updates C. Real-time transaction processing for daily operations D. Fully normalized schema design for speed

Answer: B. **Why:** OLAP serves analytical/decision-making needs over large historical datasets with complex queries and de-normalized, batch-refreshed designs — the opposite profile of OLTP.

Q6. In the Network data model compared to the Hierarchical model:

A. A child record can have only one parent, same as hierarchical B. A child record can be linked to multiple parent records, unlike hierarchical C. There is no concept of parent-child relationships D. It only supports object-oriented data

Answer: B. **Why:** The Network model generalizes the hierarchical (strict tree) model by allowing multiple-parent linkages, forming a graph-like structure.

Q7. [Trap] In a 3-tier client-server architecture, business/application logic resides primarily in:

A. The client tier B. The database tier C. A separate middle application-server tier D. It is split evenly and arbitrarily across all tiers

Answer: C. **Why:** 3-tier architecture's defining feature is separating presentation (client), business logic (application server), and data (database server) into distinct tiers, unlike 2-tier where logic sits with the client.

Q8. Which of the following operations, if executed, typically causes an implicit commit and cannot be rolled back in most RDBMSs?

A. UPDATE B. DELETE C. CREATE TABLE (DDL) D. INSERT

Answer: C. **Why:** DDL commands like CREATE TABLE auto-commit in most database systems, unlike DML commands (INSERT/UPDATE/DELETE) which can be rolled back within a transaction.

2. ER Modeling

2.1 Core Concepts

- **ER Model** (Peter Chen, 1976): conceptual/high-level design tool bridging real-world requirements and technical schema, using **Entities, Attributes, Relationships**.
- **Entity**: a real-world object distinguishable from others (concrete: person, book; abstract: course, flight reservation).
- **Entity Set**: collection of entities sharing the same attributes → rectangle in ER diagram → **table** in relational model. (An individual *entity* = a row/tuple, not shown directly in an ER diagram, since ER models schema, not instances.)

2.2 Attribute Types

Type	Meaning	Example
Simple	Cannot be subdivided	Age
Composite	Divisible into sub-parts (→ its own separate column(s) in relational model)	Name → First+Last
Single-valued	One value per entity	Aadhar No
Multivalued	Multiple values possible (double ellipse; → separate table in relational model, with main table's PK as FK)	Phone numbers, Email
Stored	Physically stored	Date of Birth
Derived	Computed from other attributes	Age (from DOB)
Descriptive	Belongs to a <i>relationship</i> , not an entity	e.g., "date of enrollment" on a Student–Course relationship

- Null value meanings: **not applicable** (value doesn't exist) vs **unknown** (missing — exists but not recorded, or genuinely not known if it exists).
- **TRAP**: A multivalued attribute is NOT stored as a comma-separated list in one column — relational conversion always needs a **separate child table** (PK of parent + the multivalued attribute, combined, as the child's PK).

2.3 Relationships & Degree

- **Relationship** = association between ≥ 2 entity sets; diamond shape in ER diagram.

Degree = number of entity sets participating in a relationship
 Unary (1 entity set, self-referential, e.g. "supervises")
 Binary (2 entity sets) <- most common
 Ternary (3 entity sets)
 N-ary (n entity sets)

2.4 Cardinality Ratios & Participation

Ratio	Meaning
1:1	≤ 1 entity in A $\leftrightarrow$ ≤ 1 entity in B
1:N	1 entity in A $\leftrightarrow$ many in B; but each B $\leftrightarrow$ ≤ 1 A
N:1	mirror of 1:N
M:N	many $\leftrightarrow$ many, both sides unrestricted

- **Participation constraint**: does existence of an entity *depend on* the relationship?
- **Total participation** (min cardinality ≥ 1): EVERY entity in the set must participate.
- **Partial participation** (min cardinality = 0): some entities may not participate at all.
- **TRAP**: Total participation $\Leftrightarrow$ **minimum cardinality ≥ 1** , not maximum. A common wrong-answer trap swaps min and max cardinality when defining total/partial participation.

2.5 Strong vs Weak Entity Sets

- **Strong entity set**: has its own primary key; tuples fully distinguishable.
- **Weak entity set**: lacks enough attributes for its own PK; has a **discriminator/partial key** (distinguishes weak entities only *within* one owner). Double rectangle in ER diagram.
- Must be linked to a **strong (owner/identifying) entity set** via an **identifying relationship** (double diamond).
- Identifying relationship is always **many-to-one** (weak → owner) and the weak entity's participation is always **total**.
- **PK of weak entity** = **PK of owner entity** $\cup$ **discriminator (partial key) of the weak entity**.

- Why weak entities exist: reflect true logical dependency, auto-delete when owner is deleted (cascading), avoid redundancy/inconsistency that would occur if data were duplicated into the owner table.

2.6 ER-to-Relational Conversion Rules

Strong entity set	-> its own table
Weak entity set	-> its own table; PK = owner's PK + discriminator
Unary relationship	-> add FK column referencing own table's PK (no new table)
Binary 1:1	-> no new table; put PK of one side as FK on the other (prefer placing FK on the side with TOTAL participation)
Binary 1:N / N:1	-> no new table; put PK of the "1" side as FK on the "N" side
Binary M:N	-> NEW table; PK = combination of both sides' PKs (as FKs)
Ternary / N-ary	-> NEW table; PK = combination of all participating entities' PKs
Multivalued attribute	-> NEW table; PK = (owner's PK + the multivalued attribute)
Composite attribute	-> a separate column per simple sub-attribute (no new table)

- TRAP:** Only **M:N (and ternary+)** relationships require a brand-new table. 1:1 and 1:N/N:1 relationships are folded into existing tables via a foreign key — a very common "how many tables are needed" numerical question hinges on this rule.

2.7 ER Traps

Trap	Cause	Fix
Fan Trap	Two or more 1:N relationships fan out from a single entity , making it impossible to determine which instance on one "many" side relates to which on the other	Restructure the model to represent the correct direct association
Chasm Trap	Two entities are connected only via a third entity with partial participation , so the model implies a path/relationship that doesn't actually exist for every occurrence	Add a direct relationship between the two entities

- TRAP:** Fan Trap arises from **1:N branching out of one entity** (ambiguous many-side matching); Chasm Trap arises from **partial participation along an indirect path** (implied but missing connectivity) — these have different root causes and different fixes; don't conflate them.

2.8 ER Diagram — Pros/Cons

- Advantages:** easy conversion to relational tables; simple, intuitive, minimal training needed.
- Disadvantages:** loss of information content (can't express all constraints), limited constraint representation, overly complex for very small projects.

Questions

Q1. A weak entity set's primary key consists of:

A. Only its own discriminator attribute B. Only the owner entity's primary key C. The owner entity's primary key combined with the weak entity's discriminator (partial key) D. No primary key is needed for weak entities

Answer: C. **Why:** A discriminator alone only distinguishes weak entities within one owner; combining it with the owner's PK makes each weak-entity tuple globally unique.

Q2. [Trap] Total participation of an entity set in a relationship means:

A. Maximum cardinality is one B. Minimum cardinality is zero C. Minimum cardinality is at least one — every entity must participate D. The relationship must be many-to-many

Answer: C. **Why:** Total participation requires min cardinality ≥ 1 (every entity occurrence participates); partial participation is min cardinality = 0.

Q3. Converting a Many-to-Many (M:N) binary relationship into the relational model requires:

A. No new table; add FK to either side B. A new table whose PK is the combination of both entities' PKs C. Modifying only the "many" side with a foreign key D. Merging both entity tables into one

Answer: B. **Why:** M:N relationships cannot be represented via a simple FK on one side (would violate the many-to-many nature); they need a bridge/junction table keyed by the combined PKs.

Q4. [Trap] A single company site has many departments and employs many staff, but the ER diagram cannot express which specific staff work in which specific department because two separate 1:N relationships fan out from the "Site" entity. This is a:

A. Chasm Trap B. Fan Trap C. Normalization anomaly D. Weak entity problem

Answer: B. **Why:** Multiple 1:N relationships emerging from one entity, making many-side associations ambiguous, is the defining signature of a Fan Trap.

Q5. A chasm trap is fixed by:

A. Adding a discriminator attribute B. Creating a direct relationship between the two entities that need to be connected C. Converting the weak entity to a strong entity D. Removing the third entity entirely

Answer: B. **Why:** Since the implied path through the third (partially-participating) entity doesn't always exist, a direct relationship restores the missing connectivity.

Q6. A multivalued attribute like "Phone Numbers" for a Customer entity is converted to the relational model as:

A. A single column storing comma-separated values B. A separate table with (Customer's PK + Phone Number) as its composite PK C. It is dropped since it cannot be represented D. Multiple separate tables, one per phone number

Answer: B. **Why:** Standard conversion rule: multivalued attributes become their own child table, PK = owner's PK combined with the attribute value.

Q7. For a binary 1:1 relationship where one side has total participation and the other has partial participation, best practice is to:

A. Always place the FK arbitrarily on either side B. Place the FK on the side with total participation C. Create a new bridging table regardless D. It cannot be converted to relational model

Answer: B. **Why:** Placing the FK on the total-participation side avoids NULL foreign keys, since every entity on that side is guaranteed to have a matching related entity.

Q8. A ternary relationship (3 entity sets) is converted into the relational model by:

A. Splitting it into three separate binary relationships always B. Creating one new table whose PK is the combination of all three participating entities' PKs C. Ignoring the third entity D. Making it a weak entity set

Answer: B. **Why:** Ternary (and higher, N-ary) relationships always require their own bridging table combining all participating entities' keys — same principle as M:N binary relationships, generalized.

Q9. [Conceptual] Why can't an ER diagram represent an individual entity directly?

A. Because entities don't exist in ER diagrams at all B. Because an ER diagram models the schema (entity SETS/types), not instances/data, and an individual entity is an instance C. Because entities are only shown in the relational model D. Because entities require a discriminator

Answer: B. **Why:** ER diagrams are schema-level (conceptual) design tools; a specific entity (a particular row of data) is an instance, which belongs to the database instance, not the schema diagram.

3. Relational Model & Functional Dependencies

3.1 Relational Model Terminology

- **RDBMS** based on Edgar F. Codd's 1970 relational model.

Term	Meaning
Relation / Table	Set of tuples
Tuple / Row / Record	One entry in a relation
Attribute / Column / Field	A named property
Domain	Set of permissible atomic (indivisible) values for an attribute
Degree / Arity	Number of attributes (columns)
Cardinality	Number of tuples (rows)

3.2 Properties of a Relation

Atomic cell values; all values in a column same type; every row unique; unique table names in a schema; unique column names; **row order insignificant; column order insignificant**.

- **TRAP**: "Order of rows/columns doesn't matter" is a defining relational property — a query result presented in a different row order is still the same relation.

3.3 Update Anomalies (motivation for Normalization)

Anomaly	Description
Insertion	Can't record one fact without also inserting unrelated/unknown data (e.g., can't add a new branch until it has an employee)
Deletion	Deleting one fact accidentally destroys other, unrelated information (e.g., deleting the last employee of a branch deletes the branch's HOD info too)
Modification (Update)	The same fact is duplicated in many rows, so updating it means editing multiple rows (miss one → inconsistency)

- These anomalies arise from mixing multiple "ideas"/themes in one table — the fix is **normalization**, using **Functional Dependencies** as the analytical tool.

3.4 Functional Dependency (FD)

alpha $\rightarrow$ beta (read: "alpha functionally determines beta")
 Formal definition: for every pair of tuples t1, t2 in relation R,
 if t1[alpha] = t2[alpha] then t1[beta] = t2[beta]

- **alpha = determinant, beta = dependent**.
- **Trivial FD**: beta $\subseteq$ alpha (always holds automatically, e.g., AB $\rightarrow$ A).
- If **K $\rightarrow$ R** (K determines every attribute of R), K is a **super key** of R.
- FD is a property of the **schema** (a design-time rule), not of one particular instance — **TRAP**: you can only *disprove* an FD from a sample instance (find 2 tuples with matching α but different β); you can never *fully prove* an FD is a true schema-level rule just by checking one instance (the instance might just accidentally satisfy it).
- Quick instance-check shortcut: FD $\alpha \rightarrow \beta$ is **violated** iff you find two tuples with equal α but different β values.

3.5 Armstrong's Axioms (sound & complete inference rules)

Reflexivity: if Y subset-of X, then X $\rightarrow$ Y (trivial FDs)
 Augmentation: if X $\rightarrow$ Y, then XZ $\rightarrow$ YZ
 Transitivity: if X $\rightarrow$ Y and Y $\rightarrow$ Z, then X $\rightarrow$ Z

Derived (secondary) rules:

Union: if X $\rightarrow$ Y and X $\rightarrow$ Z, then X $\rightarrow$ YZ
 Decomposition: if X $\rightarrow$ YZ, then X $\rightarrow$ Y and X $\rightarrow$ Z
 Pseudotransitivity: if X $\rightarrow$ Y and WY $\rightarrow$ Z, then WX $\rightarrow$ Z
 Composition: if X $\rightarrow$ Y and Z $\rightarrow$ W, then XZ $\rightarrow$ YW

- **Sound**: every FD inferred via these axioms genuinely holds in any relation state satisfying F. **Complete**: repeated application eventually derives ALL FDs implied by F (the full closure F+).

- **TRAP:** Reflexivity, Augmentation, Transitivity are the **primary/basic** axioms; Union/Decomposition/Pseudotransitivity/Composition are **derived** from them — a common "which are the 3 basic Armstrong axioms" recall question.

3.6 Attribute Closure (X^+)

X^+ = set of ALL attributes functionally determined by X (directly or by inference)
 Algorithm:
 result = X
 repeat: for each FD ($a \rightarrow b$) in F , if a subset-of result, then result = result UNION b
 until no change

- **Use 1 — Candidate key check:** X is a **super key** iff $X^+ =$ all attributes of R . X is a **candidate key** if $X^+ = R$ AND no proper subset of X also satisfies $X^+ = R$ (minimality).
- **Use 2 — Check if an FD $X \rightarrow Y$ is implied by F :** compute X^+ ; if $Y \subseteq X^+$, the FD holds (is implied).

Worked Example

$R(A,B,C,D)$, $F = \{A \rightarrow B, B \rightarrow C, AB \rightarrow D\}$. Find A^+ .

$A^+ = \{A\} \rightarrow$ apply $A \rightarrow B$: $\{A,B\} \rightarrow$ apply $B \rightarrow C$: $\{A,B,C\} \rightarrow$ apply $AB \rightarrow D$ ($AB \subseteq$ current set): $\{A,B,C,D\}$.

$A^+ = \{A,B,C,D\}$ = all of $R \Rightarrow A$ is a super key; since no smaller subset of $\{A\}$ exists, A is a candidate key.

3.7 Equivalence of Two FD Sets

F_1 and F_2 are equivalent $\Leftrightarrow F_1^+ = F_2^+$ (i.e., $F_1 \models F_2$ AND $F_2 \models F_1$)

Check by computing, for every FD in F_2 , whether its LHS-closure (under F_1) contains its RHS — and vice versa.

3.8 Minimal Cover / Canonical Cover

- **Minimal cover** = an equivalent FD set with **no redundancy**: (1) every RHS is a single attribute, (2) no extraneous attribute on any LHS, (3) no entire FD is redundant (derivable from the others).

Procedure:

1. Decompose: split every FD's RHS into single attributes (via Decomposition rule).
2. Remove extraneous LHS attributes: for FD ($AC \rightarrow B$), test if A alone already determines B (i.e., is B in $(A)^+$ computed using the current FD set?); if yes, C was extraneous — drop it.
3. Remove redundant FDs: for each FD, remove it and check if its RHS is still derivable from LHS using the REMAINING FDs; if yes, the FD itself was redundant — drop it permanently.

- **TRAP:** A minimal cover is **not always unique** — different valid orders of eliminating redundancy can yield different (but equally minimal, same-size) covers.

Worked Example (fully verified)

$F = \{AC \rightarrow BD, A \rightarrow C, B \rightarrow C, D \rightarrow C\}$ on $R(A,B,C,D)$.

Step 1 (split RHS): $AC \rightarrow B, AC \rightarrow D, A \rightarrow C, B \rightarrow C, D \rightarrow C$.

Step 2 (extraneous LHS): Test C in " $AC \rightarrow B$ ": compute A^+ using the full set $\rightarrow A \rightarrow C$ gives C , then $AC \rightarrow B$ (LHS now satisfied) gives B , then $AC \rightarrow D$ gives $D \Rightarrow A^+ = ABCD$. Since $B \in A^+$, C was extraneous $\Rightarrow$ replace $AC \rightarrow B$ with **$A \rightarrow B$** . Same logic makes $AC \rightarrow D$ become **$A \rightarrow D$** .

Set now: $\{A \rightarrow B, A \rightarrow D, A \rightarrow C, B \rightarrow C, D \rightarrow C\}$.

Step 3 (redundant whole FDs): Test $A \rightarrow C$: compute A^+ using the OTHER four FDs $\{A \rightarrow B, A \rightarrow D, B \rightarrow C, D \rightarrow C\} \rightarrow A \rightarrow B$ gives B , $A \rightarrow D$ gives D , $B \rightarrow C$ gives $C \Rightarrow A^+ = ABCD$ already contains $C \Rightarrow$ **$A \rightarrow C$ is redundant, remove it**. Testing $A \rightarrow B, A \rightarrow D, B \rightarrow C, D \rightarrow C$ similarly shows none of them are redundant.

Final minimal cover = $\{A \rightarrow B, A \rightarrow D, B \rightarrow C, D \rightarrow C\}$.

Questions

Q1. In a relation $R(A,B,C,D,E)$, the number of columns (5) is called its:

A. Cardinality B. Domain C. Degree/Arity D. Instance

Answer: C. **Why:** Degree/Arity = number of attributes (columns); Cardinality = number of tuples (rows).

Q2. [Trap] Deleting the only employee of a branch also erases the branch's name and HOD info from an unnormalized combined table. This is an example of:

A. Insertion anomaly B. Deletion anomaly C. Modification anomaly D. Referential integrity violation

Answer: B. **Why:** Losing unrelated information (branch details) as an unintended side-effect of deleting a specific record (the employee) is precisely a deletion anomaly.

Q3. [Numerical] Given $R(A,B,C,D)$, $F = \{A \rightarrow B, B \rightarrow C, AB \rightarrow D\}$. What is A^+ ?

A. $\{A,B\}$ B. $\{A,B,C\}$ C. $\{A,B,C,D\}$ D. $\{A\}$

Answer: C. **Why:** $A \rightarrow B$ gives B ; $B \rightarrow C$ gives C ; $AB \rightarrow D$ (AB now available) gives $D \Rightarrow A^+ = ABCD$.

Q4. [Conceptual] Which three are Armstrong's PRIMARY (basic) axioms?

A. Union, Decomposition, Composition B. Reflexivity, Augmentation, Transitivity C. Pseudotransitivity, Union, Reflexivity D. Augmentation,

Composition, Decomposition

Answer: B. **Why:** Reflexivity, Augmentation, and Transitivity are the three basic Armstrong axioms; Union, Decomposition, Pseudotransitivity, and Composition are all derivable secondary rules.

Q5. [Trap] A functional dependency $X \rightarrow Y$ that appears to hold on a specific sample instance of a relation:

A. Is proven to be a true schema-level FD B. May simply be a coincidence of that particular instance's data, not a guaranteed schema rule C. Can never be a valid FD D. Automatically becomes a candidate key

Answer: B. **Why:** An FD is a schema-level constraint; a sample instance can only *disprove* an FD (by finding matching- α /different- β tuples) — satisfying it in one instance doesn't prove it holds for all legal instances.

Q6. [Numerical] Find the minimal cover for $F = \{AC \rightarrow BD, A \rightarrow C, B \rightarrow C, D \rightarrow C\}$ on $R(A,B,C,D)$.

A. $\{A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow D\}$ B. $\{A \rightarrow B, A \rightarrow D, A \rightarrow C, B \rightarrow D\}$ C. $\{A \rightarrow B, A \rightarrow D, A \rightarrow C, B \rightarrow C\}$ D. $\{A \rightarrow B, A \rightarrow D, D \rightarrow C, B \rightarrow C\}$

Answer: D. **Why:** After decomposing RHS and removing the extraneous C from $AC \rightarrow B/AC \rightarrow D$ (both become $A \rightarrow B, A \rightarrow D$), the redundant FD $A \rightarrow C$ is eliminated since A already reaches C via $A \rightarrow B \rightarrow C$, leaving the minimal set $\{A \rightarrow B, A \rightarrow D, D \rightarrow C, B \rightarrow C\}$.

Q7. If X^+ (closure of X) equals the full attribute set of R, and no proper subset of X also has this property, then X is:

A. A foreign key B. A candidate key C. A trivial dependency D. A weak entity's discriminator

Answer: B. **Why:** $X^+ = R$ makes X a super key; minimality (no smaller subset also works) upgrades it specifically to a candidate key.

Q8. Two FD sets F1 and F2 are said to be equivalent when:

A. They have the same number of FDs B. $F1^+ = F2^+$ (their closures are identical) C. F1 is a subset of F2 D. They apply to different relations

Answer: B. **Why:** Equivalence is defined purely by identical closures, regardless of how many or which individual FDs each set explicitly lists.

4. Keys & Integrity Constraints

4.1 Types of Keys

- **Super Key:** any set of attributes X such that X^+ (closure) = all attributes of R . Uniquely identifies each tuple, but may contain extra/redundant attributes.
- **Candidate Key:** a *minimal* super key (no proper subset is also a super key). Also called minimal super key.
- **Prime attribute:** any attribute that is part of AT LEAST ONE candidate key. **Non-prime:** part of none.
- **Primary Key:** one candidate key chosen by the DBA to uniquely identify tuples; cannot be NULL; at most one PK per table.
- **Alternate Key:** candidate key(s) NOT chosen as primary key.
- **Composite Key:** a key made of more than one attribute (concatenated key).
- **Secondary Key:** an attribute (not necessarily unique) used to speed up search/retrieval — unlike PK, values need not be unique.

For a relation with n attributes, if EVERY attribute participates in some minimal super key in the "all attributes are prime" edge case, max possible super keys = $2^n - 1$ (any non-empty subset works only when the FULL attribute set is the sole candidate key, i.e., no smaller subset determines all attributes).

TRAP: "Super key" and "Candidate key" are often confused — every candidate key IS a super key, but not every super key is minimal (hence not every super key is a candidate key). The primary key is always a candidate key, never just any super key.

4.2 Finding All Candidate Keys (given a set of FDs)

1. Identify attributes that **never appear on the RHS of any FD** — these are "source-only" attributes and MUST belong to every candidate key (nothing else can ever derive them).
2. Compute the closure of this mandatory set. If it already equals all attributes of R , it IS the (unique) candidate key.
3. If not, try adding one attribute at a time (preferring attributes not already covered) and recompute closure until it covers all attributes, checking minimality by removing each added attribute and confirming closure shrinks.

Worked Example

$R(W, X, Y, Z)$, FDs: $X \rightarrow W$, $WZ \rightarrow XY$, $Y \rightarrow WXZ$

Attributes never on RHS of any FD: none obviously excluded — but check $Y: Y \rightarrow WXZ$, so Y alone gives $Y^+ = \{W, X, Y, Z\}$ = all attributes → **Y is a candidate key** by itself.

Check $X: X^+ = \{X, W\}$ (from $X \rightarrow W$ only) — incomplete, not a key alone.

So here Y is one candidate key; checking further combinations is needed for completeness, but Y alone already qualifies as minimal (single attribute, cannot be smaller).

4.3 Foreign Key & Referential Integrity

- **Foreign Key (FK):** an attribute (or set) in one table that refers to the candidate key (usually PK) of the same or another table, representing a relationship.
- **Referential Integrity constraint:** every non-null FK value must match an existing candidate-key value in the referenced table — no "dangling" references allowed.
- **Entity Integrity constraint:** Primary Key attributes can NEVER be NULL (distinct from referential integrity, which governs FK values).
- **On-delete / On-update actions** when the referenced row changes/is removed:

Action	Behavior
CASCADE	Automatically delete/update the dependent (child) rows too
SET NULL	Set the FK value in child rows to NULL
RESTRICT / NO ACTION	Reject the delete/update if any child row still references it

Worked Example — ON DELETE CASCADE

Table $T(A, B)$ where B is a self-referencing FK to candidate key A (e.g., an "employee → manager" style hierarchy), ON DELETE CASCADE.

Rows: (1,2), (2,3), (3,5), (5,9), (6,2). Delete the row where $A=3$.

Step 1: Delete (3,5) directly ($A=3$ matched).

Step 2 (cascade): Any row whose $B=3$ must also cascade-delete → (2,3) is removed.

Step 3 (cascade continues): Now $A=2$ is gone; any row whose $B=2$ cascades → (1,2) and (6,2) are removed.

Step 4: Now $A=1$ gone; any row with $B=1$? None. Cascade stops.

Total removed: (3,5), (2,3), (1,2), (6,2) = 4 rows. Remaining: (5,9) only, i.e., 1 row out of the original 5.

TRAP: Cascade deletes can ripple recursively through self-referencing FKs — always check every newly-deleted key value for further dependents, not just the first level.

Questions

Q1. Which statement correctly distinguishes a super key from a candidate key?

A. They are always identical B. Every candidate key is a super key, but a super key need not be minimal (candidate key) C. Every super key is a candidate key D. A table can have at most one super key

Answer: B. **Why:** Candidate key = minimal super key; a super key may contain extra, non-essential attributes and still uniquely identify tuples.

Q2. [Trap] A table has PK = {A} and a separate UNIQUE, NOT NULL column B that isn't chosen as PK. B is called:

A. A foreign key B. An alternate key C. A secondary key D. A composite key

Answer: B. **Why:** Any candidate key not selected as the primary key is an alternate key.

Q3. Entity integrity constraint specifically requires that:

A. Foreign keys always reference an existing row B. Primary key attributes can never be NULL C. All attributes must be atomic D. Every table must have a foreign key

Answer: B. **Why:** Entity integrity is about the primary key never being NULL; referential integrity (a separate rule) governs foreign key validity.

Q4. [Numerical] R(W,X,Y,Z) with FDs $X \rightarrow W$, $WZ \rightarrow XY$, $Y \rightarrow WXZ$. Which of the following is a candidate key?

A. {X} B. {W} C. {Y} D. {Z}

Answer: C. **Why:** $Y^+ = \{W,X,Y,Z\}$ (via $Y \rightarrow WXZ$) = all attributes, so {Y} alone is a (minimal) candidate key; X^+ only reaches {X,W}, incomplete.

Q5. A foreign key configured with ON DELETE SET NULL means that when the referenced row is deleted:

A. The delete is rejected if children exist B. Dependent rows are deleted too C. The FK column in dependent rows is set to NULL, and the rows themselves remain D. Nothing happens at all

Answer: C. **Why:** SET NULL preserves the dependent rows but nullifies the now-invalid reference, unlike CASCADE (which removes the rows) or RESTRICT (which blocks the operation).

Q6. [Trap] RESTRICT and NO ACTION on a foreign key both:

A. Allow the parent row deletion unconditionally B. Prevent deleting/updating a referenced row while dependent child rows still exist C. Set child FK values to NULL D. Cascade the delete automatically

Answer: B. **Why:** Both block the operation to protect referential integrity when dependents exist; they differ mainly in *when* the check fires (immediate vs. deferred to end of statement/transaction), not in the end effect for simple cases.

Q7. Which of these is true about prime attributes?

A. A prime attribute must be part of the primary key only B. A prime attribute is any attribute belonging to at least one candidate key C. Prime attributes can never be part of a foreign key D. A relation can have at most one prime attribute

Answer: B. **Why:** "Prime" is defined relative to ALL candidate keys, not just the chosen primary key — an attribute in any alternate key is also prime.

5. Normalization (1NF - BCNF)

5.1 Purpose & Progression

- Normalization** = decomposing a relation using FDs/candidate keys to eliminate redundancy and the anomalies it causes: **Insertion, Update, Deletion anomalies**.
- Progression: **1NF** → **2NF** → **3NF** → **BCNF** (each stricter than the last). Plain FD-based normalization stops at BCNF; MVD/JD are needed for 4NF/5NF (Ch.6).
- Prime attribute**: part of some candidate key. **Non-prime**: part of none. **Partial dependency**: a non-prime attribute depends on a proper subset of a (composite) candidate key. **Total dependency**: depends on the whole key. **Transitive dependency**: a non-prime attribute functionally depends on another non-prime attribute (via a chain, not directly on a key).

5.2 Definitions

Form	Requirement
1NF	All attribute values atomic (no multivalued/composite attributes); unique rows; a PK exists.
2NF	In 1NF + no partial dependency of any non-prime attribute on a candidate key. (Automatically satisfied if every candidate key is a single attribute — partial dependency needs a composite key to even exist.)
3NF	In 2NF + no transitive dependency . Equivalent "direct" definition (exam-preferred): for every FD $X \rightarrow Y$ in F, either X is a super key, OR Y is a prime attribute .
BCNF	For every FD $X \rightarrow Y$ in F, X must be a super key — no prime-attribute exception (strictly stronger than 3NF).

TRAP: The 3NF "OR Y is prime" escape clause is exactly what BCNF removes — a relation can be in 3NF (satisfying that OR) yet fail BCNF (X still not a super key).

5.3 Useful Shortcuts (memorize for quick MCQ elimination)

- A relation with **only 2 attributes** is always in BCNF.
- If R has **no non-trivial FD** at all, R is automatically in BCNF.
- If R's candidate key is a **single attribute (simple key)**, R is always at least in **2NF** (no partial dependency possible) — but may still fail 3NF/BCNF.
- If **every attribute of R is prime**, R is always at least in **3NF** (the "Y is prime" clause always rescues it) — but may still fail BCNF.
- If R is in **3NF** and has **only simple (single-attribute) candidate keys**, R is guaranteed to also be in **BCNF**.

5.4 Worked Examples (highest normal form given FDs) — verified by hand

Example 1 — stops at 1NF (classic GATE pattern)

$R(A,B,C,D,E,F,G,H)$, $F = \{CH \rightarrow G, A \rightarrow BC, B \rightarrow CFH, E \rightarrow A, F \rightarrow EG\}$
 D never appears anywhere in F (no FD produces or uses D) → D must be in every candidate key.
 $\{A,D\}^+ : A \rightarrow BC$ adds B,C; $B \rightarrow CFH$ adds F,H; $F \rightarrow EG$ adds E,G → closure = all 8 attributes. $\{A\}$ alone only reaches $\{A,B,C,E,F,G,H\}$ (misses D).
 So **candidate key = $\{A,D\}$** (unique, composite).
 Prime = $\{A,D\}$. Non-prime = $\{B,C,E,F,G,H\}$.
 Check 2NF: $A \rightarrow BC$ is a dependency of non-prime B,C on just **$\{A\}$** , a proper subset of the key $\{A,D\}$ → **partial dependency exists** → 2NF fails.
Result: 1NF only (not even 2NF).

Example 2 — 2NF but not 3NF

$R(A,B,C,D)$, $F = \{A \rightarrow BC, C \rightarrow D\}$
 $A^+ = \{A,B,C,D\}$ ($A \rightarrow BC$, then $C \rightarrow D$) = all attributes → **$\{A\}$ is the candidate key** (single attribute).
 Since the key is a single attribute, no partial dependency is possible → **2NF holds automatically**.
 Check 3NF: for $C \rightarrow D$ — is C a super key? $C^+ = \{C,D\}$ only → No. Is D prime? No (only A is prime). → violates 3NF.
Result: 2NF, not 3NF.

Example 3 — 3NF but not BCNF (the classic textbook case)

$R(A,B,C)$, $F = \{AB \rightarrow C, C \rightarrow A\}$
 $\{AB\}^+ = \{A,B,C\}$ → superkey, minimal (A alone, B alone insufficient) → **candidate key $\{A,B\}$** .
 Also check $\{BC\}^+ = \{B,C,A\}$ (via $C \rightarrow A$) = all → **$\{B,C\}$ is also a candidate key**.
 Prime attributes = $\{A,B,C\}$ (all attributes are prime — each is in some candidate key!). Non-prime = none.
 Check 3NF: $AB \rightarrow C$ (AB is a superkey, fine). $C \rightarrow A$: is C a superkey? No ($\{C\}^+ = \{C,A\}$, missing B). Is A prime? **Yes** → satisfies the 3NF "Y is prime" clause.
Result: 3NF holds. Check BCNF: $C \rightarrow A$ has $X=C$ which is NOT a superkey, and BCNF allows no exception → **BCNF fails**.
Result: 3NF, not BCNF.

Example 4 — full BCNF

$R(A,B,C)$, $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$

Every single attribute's closure reaches all three ($A^+ = B^+ = C^+ = \{A,B,C\} \rightarrow \{A\}, \{B\}, \{C\}$ are all individually candidate keys.

Every FD's LHS (A , B , or C) is therefore always a superkey by itself.

Result: BCNF holds (every determinant is a key — the strictest requirement is met).

Questions

Q1. A relation with FD set having only a single-attribute candidate key can never violate:

A. 3NF B. BCNF C. 2NF D. 1NF

Answer: C. Why: Partial dependency (the only 2NF violation) requires a *composite* candidate key; a single-attribute key makes 2NF automatic.

Q2. [Numerical] $R(A,B,C,D)$ with $F = \{A \rightarrow BC, C \rightarrow D\}$. What is the highest normal form satisfied?

A. 1NF B. 2NF C. 3NF D. BCNF

Answer: B. Why: $\{A\}$ is the sole candidate key (single attribute $\Rightarrow$ automatic 2NF); but $C \rightarrow D$ violates 3NF since C is not a superkey and D is not prime.

Q3. [Trap] $R(A,B,C)$ with $F = \{AB \rightarrow C, C \rightarrow A\}$. This relation is:

A. In BCNF B. In 3NF but not BCNF C. In 2NF but not 3NF D. Not even in 1NF

Answer: B. Why: Candidate keys are $\{A,B\}$ and $\{B,C\}$; all attributes are prime, so $C \rightarrow A$ survives 3NF via the "Y is prime" clause, but fails BCNF since C is not a superkey.

Q4. Which condition, by itself, guarantees that a 3NF relation is automatically also in BCNF?

A. The relation has more than 3 attributes B. Every candidate key of the relation is a single (simple) attribute C. The relation has no foreign keys D. The relation is fully normalized to 5NF

Answer: B. Why: With only simple candidate keys, the 3NF "prime attribute" exception can never rescue a genuine BCNF violation the way it can with composite keys — 3NF + simple keys $\Rightarrow$ BCNF.

Q5. A table $R(A, B)$ with exactly two attributes and any FD between them is always in:

A. 1NF only B. 2NF only C. 3NF only D. BCNF

Answer: D. Why: A 2-attribute relation is always in BCNF — the determinant of any FD $A \rightarrow B$ or $B \rightarrow A$ must be a candidate key by elimination.

Q6. [Numerical] $R(A,B,C,D,E,F,G,H)$ with $F = \{CH \rightarrow G, A \rightarrow BC, B \rightarrow CFH, E \rightarrow A, F \rightarrow EG\}$. Highest normal form?

A. 1NF, not 2NF B. 2NF, not 3NF C. 3NF, not BCNF D. BCNF

Answer: A. Why: D appears in no FD, so the sole candidate key is $\{A,D\}$; $A \rightarrow BC$ is a partial dependency (B,C non-prime, depend only on $A \subset \{A,D\}$) — fails 2NF, so R sits only in 1NF.

Q7. A relation R has no non-trivial functional dependency at all. R is:

A. Not necessarily in any normal form B. Automatically in BCNF C. Automatically only in 1NF D. Automatically in 2NF but never BCNF

Answer: B. Why: With no non-trivial FDs, there is nothing to violate any of the normal-form rules (all conditions are vacuously true) — R is trivially in BCNF.

Q8. Which of the following is the correct "direct" (LHS/RHS) definition of 3NF used for fast exam elimination?

A. No composite candidate keys allowed B. For every FD $X \rightarrow Y$, X must be a superkey C. For every FD $X \rightarrow Y$, either X is a superkey or Y is a prime attribute D. Every attribute must be a candidate key

Answer: C. Why: This is the standard 3NF test; BCNF (option B) drops the "Y is prime" exception, making it strictly stronger.

6. Decomposition, Multivalued Dependency & 5NF

6.1 Why Decompose? Two Desired Properties

When normalization splits R into $R_1, R_2, \dots, R_n$, two properties are checked:

1. **Lossless (Non-Additive) Join Decomposition** — mandatory, non-negotiable.
2. **Dependency-Preserving Decomposition** — desirable, but sometimes sacrificed.

6.2 Lossless Join Decomposition

Decomposition of R into R_1, R_2 is **LOSSY** if: $R_1 \text{ JOIN } R_2 \supset R$ (produces spurious/extra tuples)
 Decomposition of R into R_1, R_2 is **LOSSLESS** if: $R_1 \text{ JOIN } R_2 = R$ (reconstructs R exactly)

Test using FDs — for R decomposed into R_1, R_2 , ALL of the following must hold:

1. $\text{Att}(R_1) \cup \text{Att}(R_2) = \text{Att}(R)$ (every attribute lands in at least one piece).
2. $\text{Att}(R_1) \cap \text{Att}(R_2) \neq \emptyset$ (they must share at least one common attribute).
3. The common attribute set must be a **key for at least one** of R_1 or R_2 :

$\text{Att}(R_1) \cap \text{Att}(R_2) \rightarrow \text{Att}(R_1)$ OR $\text{Att}(R_1) \cap \text{Att}(R_2) \rightarrow \text{Att}(R_2)$

TRAP: Sharing a common attribute is NOT enough by itself — that common attribute (or attribute set) must *functionally determine* one entire side of the decomposition, or the join will produce spurious tuples.

Worked Example

$R(A,B,C), F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$, decomposed into $R_1(A,B)$ and $R_2(B,C)$.

Common attribute = $\{B\}$. Does $B \rightarrow A$ hold? Via $B \rightarrow C \rightarrow A$ (transitivity) — yes. So B determines all of R_1 's attributes $(A,B) \rightarrow$ **condition 3 satisfied** $\rightarrow$ **lossless**.

6.3 Dependency-Preserving Decomposition

Decomposition into $R_1 \dots R_n$ (with projected FD sets $F_1 \dots F_n$) is dependency-preserving iff:
 $\{F_1 \cup F_2 \cup \dots \cup F_n\}^+ = F^+$ (the union of projected FDs, closed, equals the original closure)

- **BCNF decomposition is ALWAYS lossless, but is NOT always dependency-preserving.**
- **TRAP:** This is one of the most commonly tested DBMS facts — students assume "properly normalized $\Rightarrow$ both properties guaranteed." BCNF only guarantees losslessness; 3NF decomposition (via the synthesis algorithm) guarantees BOTH lossless AND dependency-preserving, which is why 3NF is sometimes deliberately preferred over BCNF in practice.

Worked Example (same relation as above)

$R(A,B,C), F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\} \rightarrow R_1(A,B), R_2(B,C)$.

F_1 (projected onto A,B): $A \rightarrow B$ holds; $B \rightarrow A$ also holds (derivable, since $B \rightarrow C \rightarrow A$ stays within... wait, C isn't in R_1 , but $B \rightarrow A$ is still a valid consequence of F^+ expressed using only R_1 's attributes) $\rightarrow F_1 = \{A \rightarrow B, B \rightarrow A\}$.

F_2 (projected onto B,C): $B \rightarrow C$ holds; $C \rightarrow B$ holds (via $C \rightarrow A \rightarrow B$) $\rightarrow F_2 = \{B \rightarrow C, C \rightarrow B\}$.

Union $F_1 \cup F_2 = \{A \rightarrow B, B \rightarrow A, B \rightarrow C, C \rightarrow B\}$. Does this imply $C \rightarrow A$ (needed to match F^+)? $C \rightarrow B, B \rightarrow A \Rightarrow C \rightarrow A$ by transitivity. **Yes, preserved.**

This decomposition is both lossless AND dependency-preserving.

6.4 Multivalued Dependency (MVD) & 4NF

- **MVD notation:** $X \twoheadrightarrow Y$ — for every value of X , there exists a set of values of Y , *independent* of the other attributes in R .
- **Trivial MVD:** $X \twoheadrightarrow Y$ is trivial if $Y \subseteq X$, or $X \cup Y =$ all attributes of R .
- If a normal FD $X \rightarrow Y$ holds, then the MVD $X \twoheadrightarrow Y$ also trivially holds (FD $\Rightarrow$ MVD, not the reverse).
- **Why MVDs matter:** a relation can be in BCNF (no non-trivial FDs at all) yet still have massive redundancy caused purely by two *independent* multivalued facts about the same key — this is exactly what 4NF fixes.

Classic Example

Student($S_name, Club_name, P_no$) where a student can be in multiple clubs AND have multiple phone numbers — completely independently of each other.

MVDs: $S_name \twoheadrightarrow Club_name$, and $S_name \twoheadrightarrow P_no$.

No FD holds at all (so the relation is vacuously in BCNF) — yet every (club, phone) combination must be repeated for consistency, causing redundancy.

Fix: decompose into $(S_name, Club_name)$ and (S_name, P_no) — each MVD becomes trivial in its own smaller table.

4NF definition: R is in 4NF iff it is in BCNF AND has **no non-trivial MVD**.

TRAP: A relation can satisfy BCNF perfectly (zero FD violations) and still fail 4NF purely due to independent multivalued attributes — "BCNF" does NOT imply "4NF."

6.5 Fifth Normal Form (5NF / PJNF)

- **5NF:** R is in 4NF AND every non-trivial **join dependency (JD)** is implied by the candidate keys (i.e., R cannot be losslessly decomposed into smaller pieces except via its keys).
- Targets redundancy from **three-way (or higher) circular relationships** that a 2-table decomposition cannot fix, but a 3-table decomposition can.

Classic Example — Supplier–Part–Project

A relation SPJ(Supplier, Part, Project) where the business rule is: "if S supplies P, S works with J, and J uses P, the combination is valid" — this is a genuine 3-way constraint, not fully captured by any 2 of the 3 pairwise relationships alone.

5NF decomposition: split into three binary relations — SP(Supplier,Part), SJ(Supplier,Project), PJ(Part,Project).

Rejoining $SP \bowtie SJ \bowtie PJ$ reconstructs the valid combinations losslessly — this 3-way reconstruction requirement is exactly a **non-trivial join dependency**, and 5NF says: decompose until no such non-trivial JD remains unless it's implied by the keys.

Questions

Q1. For a decomposition of R into R1 and R2 to be lossless, the common attribute set $\text{Att}(R1) \cap \text{Att}(R2)$ must:

- A. Be non-empty only B. Be a candidate key of R C. Functionally determine all attributes of R1 or all attributes of R2 D. Contain the primary key of R1 only

Answer: C. Why: The common attributes must act as a determinant (key) for at least one of the two decomposed relations — this is what guarantees no spurious tuples on rejoining.

Q2. [Trap] "A BCNF decomposition is always both lossless and dependency-preserving." This statement is:

- A. True B. False — BCNF decomposition is always lossless, but not always dependency-preserving C. False — BCNF decomposition is dependency preserving but not always lossless D. False on both counts, always

Answer: B. Why: Losslessness is guaranteed by the BCNF decomposition algorithm; dependency preservation is not, which is a classic real-world trade-off (3NF synthesis is used instead when preservation matters more).

Q3. A Multivalued Dependency $X \twoheadrightarrow Y$ is called trivial when:

- A. X and Y are both empty B. $Y \subseteq X$, or $X \cup Y$ equals all attributes of R C. X is a candidate key D. Y is a prime attribute

Answer: B. Why: These are exactly the two cases where the MVD carries no extra constraint beyond what's structurally guaranteed.

Q4. [Trap] A relation is confirmed to be in BCNF (no FD violations whatsoever). Can it still violate 4NF?

- A. No, BCNF always implies 4NF B. Yes — independent multivalued attributes can cause redundancy even with zero FDs present C. No, 4NF is weaker than BCNF so it's automatically satisfied D. Only if the relation has a composite key

Answer: B. Why: MVD-based redundancy is a separate phenomenon from FD-based redundancy; a relation can be a "perfect" BCNF design yet still redundantly repeat combinations of two independent multivalued facts — exactly what 4NF targets.

Q5. The Supplier-Part-Project example is used to illustrate the need for:

- A. 2NF B. BCNF C. 4NF D. 5NF

Answer: D. Why: It demonstrates a genuine three-way (non-trivial) join dependency that requires decomposition into three binary relations — the defining case for 5NF.

Q6. [Numerical] $R(A,B,C)$, $F=\{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$, decomposed into $R1(A,B)$, $R2(B,C)$. Is this decomposition lossless?

- A. Yes, since $B \rightarrow A$ is derivable, making B a key of R1 B. No, because C is missing from R1 C. No, since the intersection is empty D. Cannot be determined

Answer: A. Why: $\text{Att}(R1) \cap \text{Att}(R2) = \{B\}$; $B \rightarrow A$ holds (via $B \rightarrow C \rightarrow A$ transitively), so B determines all of R1 — satisfying the lossless-join condition.

Q7. Why is dependency preservation described as "desirable but sometimes sacrificed"?

- A. It is always impossible to achieve B. Enforcing certain FDs after BCNF decomposition may require an expensive join to check, so it's occasionally traded off for a lossless-only decomposition C. It only matters for 1NF relations D. Because MVDs override FDs

Answer: B. Why: Some FDs may span multiple decomposed relations after BCNF normalization, making them impossible to check without rejoining — hence BCNF sometimes trades away dependency preservation for guaranteed losslessness.

7. File Organization, Indexing & B-Trees

7.1 File Organization

- **Ordered (sorted) file:** supports **binary search** (only on the field the file is sorted by); insertion/deletion is costly (needs reorganization).
- **Unordered (heap) file:** new records appended at the end; only **linear search** possible; insertion/deletion is cheap (no reorganization).
- **TRAP:** Binary search on an ordered file works only when searching on the field it is physically sorted by — searching any other field still degrades to linear search.

7.2 Indexing Basics

- **Index** = auxiliary access structure providing a secondary access path without touching the physical placement of main-file records.
- Index file record has just 2 columns: **key + block pointer** — far smaller than the main file, so it is always kept **sorted** (regardless of whether the main file is sorted), enabling binary search on the index itself.
- Index can be built on **any** attribute (key or non-key); multiple index files can exist for one main file.

```
Blocking factor (bfr) = floor(Block size / Record size)
Number of blocks (b) = ceil(Number of records / bfr)
Block accesses (unordered file) = O(n) [n = number of blocks; linear search]
Block accesses (ordered file / single-level index, binary search) = ceil(log2(n)) + 1 [+1 for the final data-block access]
```

- **Dense index:** one entry per *distinct search-key value* (not per record) in the main file.
- **Sparse index:** entries for only *some* records — requires the main file to be ordered on that key.

7.3 Types of Indexing

Type	Main file order	Index density	Entries in index file
Primary	Ordered on primary key	Sparse	= number of blocks in main file (1 per block, via anchor/first record)
Clustering	Ordered on a non-key attribute	Sparse or dense	= number of distinct values of that attribute
Secondary	Main file order irrelevant	Dense	= number of records in main file (1 per record)
Multilevel	—	—	Recursively indexes the index itself until one block fits in memory

- **TRAP:** Primary indexing works **only on the ordering key** and is sparse; Secondary indexing can be on any (key/non-key) attribute and is always **dense** — mixing these two up is a classic exam trap.

Worked Example (Primary vs Secondary Indexing + Multilevel Benefit)

Given: $r = 30,000$ records, Block size $B = 1024$ B, Record size $R = 100$ B (unspanned),
key field $V = 9$ B, block pointer $P = 6$ B.

Main file:

```
bfr = floor(1024/100) = 10 records/block
b   = ceil(30000/10) = 3,000 blocks
```

PRIMARY INDEX (sparse, 1 entry per block of main file):

```
Index entry size = V+P = 15 B => bfri = floor(1024/15) = 68 entries/block
Index entries = 3,000 (= number of main-file blocks)
Index blocks (level 1) = ceil(3000/68) = 45
-- Single-level binary search: ceil(log2(45)) + 1 = 6 + 1 = 7 block accesses
-- Add level 2: ceil(45/68) = 1 block (fits in memory)
-- Multilevel (2-level) direct access: levels + 1 = 2 + 1 = 3 block accesses
```

SECONDARY INDEX (dense, 1 entry per record):

```
Index entries = 30,000 (= number of records)
Index blocks (level 1) = ceil(30000/68) = 442
-- Single-level binary search: ceil(log2(442)) + 1 = 9 + 1 = 10 block accesses
-- Level 2: ceil(442/68) = 7 blocks; Level 3: ceil(7/68) = 1 block (fits in memory)
-- Multilevel (3-level) direct access: levels + 1 = 3 + 1 = 4 block accesses
```

TRAP: Multilevel indexing turns an $O(\log_2 n)$ binary-search cost into an $O(\#levels)$ direct-access cost — always dramatically fewer block accesses once the index itself grows large (7→3 and 10→4 above).

7.4 B-Trees & B+ Trees

- **Why not a plain BST/AVL tree for disk indexes?** Disk I/O (block access) is the bottleneck, not comparisons — B/B+ trees maximize **fan-out** (children per node) so the tree height (and thus disk accesses) stays very small even for millions of records.

- **B-tree of order m** (m-way search tree):

Node	Max children	Min children	Max keys	Min keys
Root	m	0 (or 2 if it has any children)	m – 1	1
Internal (non-root)	m	$\lceil m/2 \rceil$	m – 1	$\lceil m/2 \rceil - 1$
Leaf	0	0	m – 1	$\lceil m/2 \rceil - 1$

- All leaves are on the **same level** (perfectly height-balanced) — this guarantees worst-case search time stays $O(\log n)$.

B-Tree vs B+ Tree

Feature	B-Tree	B+ Tree
Data pointers	At every level (internal + leaf)	Only at leaf level
Internal nodes	Store keys + data pointers	Store only keys (used purely for routing)
Leaf nodes	Normal leaves	Linked together (sequence set) for fast ordered range scans
Range queries	Poor — no way to traverse keys sequentially	Excellent — just follow leaf-level linked list
Redundant keys	No	Yes (a key may appear in both an internal node and a leaf)
Preferred for	—	Actual DB indexing (used far more in practice)

- **TRAP:** Because B+ tree internal nodes hold no data pointers, they fit **more keys per node** than a B-tree of the same order — meaning B+ trees usually achieve a **smaller height** for the same data, on top of their range-query advantage. This is why virtually all real DB indexes (and file systems) use B+ trees, not plain B-trees.

Insertion / Deletion Rules (conceptual)

- **Insertion (both):** insert in sorted position; if a node **overflows** (exceeds $m-1$ keys), split it — median key moves up to the parent (in B+ tree, the median is *copied* up, not removed from the leaf level); splitting can cascade up to the root, adding a new level.
- **Deletion (both):** if removal causes a node to **underflow** (fall below $\lceil m/2 \rceil - 1$ keys), first try to **borrow** a key from an adjacent sibling (via the parent); if siblings are also at minimum, **merge** with a sibling — this can cascade up and even reduce the tree's height.
- Empirically, after many random insert/delete operations, B-tree/B+ tree nodes stabilize at about **69% full**.

Questions

Q1. Which type of indexing must always be dense (one entry per record), regardless of whether it is built on a key or non-key attribute?

A. Primary B. Clustering C. Secondary D. Multilevel

Answer: C. **Why:** Secondary indexing always has as many entries as there are records in the main file (dense); primary indexing is sparse (1 entry per block) and requires the main file to be sorted on that key.

Q2. [Numerical] A file has 2,000 blocks in its main file, and a single-level sparse primary index is built on it. Using binary search on the index followed by one data-block access, how many total block accesses are needed in the worst case?

A. 10 B. 11 C. 12 D. 2000

Answer: C. **Why:** $\text{ceil}(\log_2(2000)) + 1 = 11 + 1 = 12$.

Q3. [Trap] Why is a B+ tree generally preferred over a plain B-tree for actual database indexing?

A. B+ trees use less memory per node B. B+ trees are not balanced, so they are faster C. B+ tree leaves are linked, enabling efficient range queries, and internal nodes (holding only keys) achieve higher fan-out D. B-trees cannot store more than 3 levels

Answer: C. **Why:** B+ tree internal nodes carry no data pointers (only routing keys), allowing more keys per node (lower height) and, crucially, their linked leaf level supports fast sequential range scans — something a B-tree cannot do efficiently.

Q4. In a B-tree of order $m=5$, what is the minimum number of keys allowed in a non-root internal node?

A. 1 B. 2 C. 3 D. 4

Answer: B. **Why:** $\text{Min keys} = \lceil m/2 \rceil - 1 = \lceil 2.5 \rceil - 1 = 3 - 1 = 2$.

Q5. In a B-tree of order $m=5$, what is the maximum number of keys a node (any node) can hold?

A. 3 B. 4 C. 5 D. 6

Answer: B. **Why:** $\text{Max keys} = m - 1 = 5 - 1 = 4$ for every node type (root, internal, leaf).

Q6. When a B+ tree leaf node overflows during insertion, the median key is:

A. Deleted B. Moved up to the parent and removed from the leaf C. Copied up to the parent while also remaining in the (new) leaf D. Ignored

Answer: C. **Why:** In a B+ tree, since only leaves hold actual data pointers, the median key must remain at the leaf level (retained/duplicated) even as its copy is pushed up to route searches at the internal level — unlike a B-tree, where the median is fully moved up.

Q7. [Trap] A junior developer claims: "B-trees are better than B+ trees for database indexes because B-trees can locate data at internal nodes without reaching a leaf." What is the flaw?

A. This is entirely correct B. B-trees don't have internal nodes C. While technically true for point lookups, this comes at the cost of no ordered leaf-linkage, making range queries (a very common DB operation) far slower than in B+ trees — hence B+ trees are still preferred in practice D. Only B-trees are balanced

Answer: C. **Why:** The point-lookup argument is real but minor compared to the huge practical cost of poor range-query support, which is why virtually all production databases use B+ trees for indexing.

Q8. What happens when deletion causes a B+ tree node to underflow and no sibling has a key to spare?

A. Nothing, underflow is ignored B. The node is merged with a sibling, potentially propagating up and reducing tree height C. The tree becomes

unbalanced permanently D. A new root is always created

Answer: B. **Why:** Merging propagates upward if the parent then also underflows, and can reduce the number of levels in the tree — the mirror-image process of insertion-triggered splits.

8. Relational Algebra

8.1 Query Languages Overview

Feature	Relational Algebra (RA)	Relational Calculus (RC)
Type	Procedural (what + how)	Non-procedural / declarative (what only)
Basis	Set theory, operators on relations	Mathematical logic (predicates)
Executed directly?	No — theoretical foundation for SQL	No — theoretical foundation for SQL

- SQL is practical and blends both procedural and declarative elements; RA/RC are the theoretical basis SQL is built on (never run directly on a real DBMS).
- Every RA operator takes 1–2 relations as input and returns exactly **one unnamed relation** as output — this closure property lets operators be nested/composed.
- TRAP:** RA is **set-based**, so it does **not** consider duplicates by default — the same query in SQL (which uses bag/multiset semantics) can return a different number of rows.

8.2 Fundamental (Basic) Operators

Operator	Symbol	Arity	Eliminates duplicates?
Select	σ (sigma)	Unary	No — only removes rows via a condition
Project	Π (pi)	Unary	Yes — by default
Union	$\cup$	Binary	Yes (set union)
Set Difference	$-$	Binary	Yes
Cartesian Product	$\times$	Binary	No
Rename	ρ (rho)	Unary	—

- Select (σ):** $\sigma_{\text{condition}}(R)$ — horizontal filter (chooses rows). Degree of result = degree of R; $0 \leq \text{cardinality}(\text{result}) \leq \text{cardinality}(R)$. σ is **commutative**: $\sigma_1(\sigma_2(R)) = \sigma_2(\sigma_1(R))$.
- Project (Π):** $\Pi_{\text{col1, col2}}(R)$ — vertical filter (chooses columns), **eliminates duplicate rows** by default. **Not commutative** with itself in the sense that column order in the subscript matters for the output schema.
- TRAP:** Select removes **rows only**, never eliminates duplicates; Project removes **columns AND** eliminates duplicate rows — a very frequently tested distinction.

8.3 Set Operators & Cartesian Product

Union-compatibility requirement for $r \cup s$, $r - s$, $r \cap s$:
Same degree (arity) AND matching attribute domains (same order).

Union: $r \cup s$; $\text{Deg}(r \cup s) = \text{Deg}(r)$; $\max(|r|, |s|) \leq |r \cup s| \leq |r| + |s|$
 Set diff: $r - s$; $0 \leq |r - s| \leq |r|$
 Intersection: $r \cap s = r - (r - s)$ [derived, not fundamental] $0 \leq |r \cap s| \leq \min(|r|, |s|)$
 Cartesian: $r \times s$; $|r \times s| = |r| * |s|$; $\text{Deg}(r \times s) = \text{Deg}(r) + \text{Deg}(s)$

- TRAP:** Union/Difference/Intersection all require **union-compatible** relations (same degree + matching domains); Cartesian Product has **no such restriction** — it pairs every tuple of r with every tuple of s regardless of schema.
- Intersection is a **derived** operator (expressible via difference) — it adds no new power to RA, just convenience.

8.4 Joins

Join hierarchy:
Join

```
-- Inner Join
|  |-- Theta Join (any comparison operator: =, <, <=, !=, >, >=)
|  |  |-- Equi Join (theta join restricted to "=" only)
|  |  |-- Natural Join (equi-join on ALL common attribute names, duplicate column dropped)
|-- Outer Join
|  |-- Left Outer Join (all of R1, unmatched R2 side padded with NULL)
|  |-- Right Outer Join (all of R2, unmatched R1 side padded with NULL)
|  |-- Full Outer Join (all of both, unmatched sides padded with NULL)
```

- **Natural Join ($\bowtie$):** implicit equality condition on all commonly-named attributes; the shared attribute appears **only once** in the result (unlike a plain Cartesian product + selection, which would keep both copies).
- **Theta Join ($R \bowtie_{\theta} S$):** general condition $A_i \theta B_j$; result has $n+m$ attributes (both copies of any shared-name columns kept, distinguished via relation-name prefix).
- **Equi Join:** theta join using **only "="** as the comparison operator.
- **TRAP:** Natural Join is really "Equi Join + drop duplicate column"; if you only apply an equi-join condition without dropping the duplicate attribute, that's technically just a theta/equi join, NOT a natural join.
- **Outer Joins** preserve unmatched tuples (pad with NULL) instead of discarding them like an inner/natural join would.

8.5 Rename Operator

```
rho_X(E)           -- rename relation resulting from expression E to X
rho_X(A1,A2,...,An)(E)  -- rename relation to X AND rename its attributes to A1..An
```

- Does **not** modify the original table in the database — it creates a renamed reference/copy of the query result, used for self-joins or giving a query result a usable name for further composition.

8.6 Division Operator ($\div$)

```
R(Z) / S(X)   where X subset-of Z, Y = Z - X
Formula: R / S = Pi_Y(R) - Pi_Y[ (Pi_Y(R) x S) - Pi_Y,X(R) ]
```

- **Meaning:** returns values of Y in R that are associated with **every** value of X in S ("Y-values related to ALL X-values").
- Recognition Pattern: if a query says **"find X who are related to ALL Y"** (e.g., "find students enrolled in ALL courses," "find suppliers who supply ALL parts") → think **Division**.
- Division is a **derived** operator (built from Project, Cartesian Product, and Set Difference) — not fundamental, but essential for "for-all" style queries that would otherwise need deeply nested RA expressions.

Questions

Q1. [Trap] Which relational algebra operator eliminates duplicate rows in its result by default?

A. Select (σ) B. Project (Π) C. Cartesian Product ($\times$) D. Theta Join

Answer: B. **Why:** Projection removes columns and, being set-based, automatically drops any resulting duplicate rows; Selection only filters rows and does not deduplicate.

Q2. [Numerical] Relation R has 5 tuples and degree 3; Relation S has 4 tuples and degree 2. What is the degree and cardinality of $R \times S$?

A. Degree 5, Cardinality 9 B. Degree 6, Cardinality 9 C. Degree 5, Cardinality 20 D. Degree 6, Cardinality 20

Answer: C. **Why:** Degree($R \times S$) = Degree(R) + Degree(S) = 3 + 2 = 5; Cardinality = $|R| \times |S| = 5 \times 4 = 20$.

Q3. For $R \cup S$ to be a valid relational algebra operation, R and S must:

A. Have the same number of tuples B. Be union-compatible (same degree and matching attribute domains) C. Share at least one common attribute name D. Have identical primary keys

Answer: B. **Why:** Union (like Set-Difference and Intersection) requires union-compatibility; unlike Cartesian Product or Join, matching attribute names is not itself the requirement — matching arity and domains is.

Q4. Which join preserves all tuples from BOTH relations, padding unmatched sides with NULL?

A. Natural Join B. Left Outer Join C. Right Outer Join D. Full Outer Join

Answer: D. **Why:** Full Outer Join = Left Outer Join + Right Outer Join combined — every tuple from both relations appears, matched or not.

Q5. A query asks: "Find the names of suppliers who supply ALL parts in the Parts table." Which RA operator is the natural fit?

A. Natural Join B. Division C. Set Difference D. Cartesian Product

Answer: B. **Why:** "Related to ALL values" is the textbook signal for the Division operator.

Q6. [Trap] A student writes a query joining Employee and Department on `Employee.dept_id = Department.dept_id` but keeps both `dept_id` columns in the output. This is best classified as:

A. A Natural Join B. An Equi Join (not a natural join, since the duplicate column wasn't dropped) C. A Left Outer Join D. Invalid RA syntax

Answer: B. **Why:** Natural Join specifically drops the duplicate shared-name column; keeping both copies makes this only an Equi Join (a type of Theta Join), not a Natural Join.

Q7. What is the correct relationship between Intersection and the fundamental RA operators?

A. Intersection is fundamental and cannot be derived B. Intersection = $R - (R - S)$, a derived operator adding no new expressive power C. Intersection requires a Cartesian Product D. Intersection is only valid for numeric attributes

Answer: B. **Why:** Set-Intersection is expressible purely via Set-Difference, so it's classified as a derived (not fundamental) operator.

Q8. [Numerical] R has attributes (A,B,C) with 100 tuples; S has attributes (B,C) with 30 distinct (B,C) combinations appearing across all of R's groups. After computing $R \div S$, which attribute(s) remain in the result?

A. A, B, C B. B, C only C. A only D. None

Answer: C. **Why:** For $R(Z) \div S(X)$ where $X \subseteq Z$, the result contains only the attributes $Y = Z - X$ — here $Z = \{A, B, C\}$, $X = \{B, C\}$, so $Y = \{A\}$.

Q9. Which of the following is NOT a valid form of outer join?

A. Left Outer Join B. Right Outer Join C. Full Outer Join D. Cartesian Outer Join

Answer: D. **Why:** The three valid outer join forms are Left ($\ltimes$), Right ($\rtimes$), and Full ($\bowtie$) — "Cartesian Outer Join" is not a defined RA operation.

9. SQL — Basics, Select/Where, DDL/DML/DCL/TCL

9.1 What SQL Is

- SQL = declarative language for RDBMS (originated as "Sequel" in IBM's System R, 1970s; standardized as SQL-86 by ANSI/ISO, evolved through SQL-92, SQL:1999, ... SQL:2023).
- Built on **Relational Algebra** (procedural) and **Tuple Relational Calculus** (non-procedural) — SQL itself is declarative (says WHAT, not HOW).
- Sub-languages: **DDL** (structure), **DML** (data), **DCL** (permissions), **TCL** (transactions), **DQL** (SELECT, sometimes classed under DML).

9.2 Basic Query Structure

```
SELECT A1, A2, ..., An
FROM r1, r2, ..., rm
WHERE P;
```

- A query takes 1+ tables as input, always produces exactly **one unnamed (anonymous) output table**.
- Only **SELECT** and **FROM** are **mandatory**; omitted WHERE = condition treated as true for all rows.
- Logical order of execution: FROM → WHERE → SELECT** (not the written top-to-bottom order!) — this ordering matters a lot once GROUP BY/HAVING enter the picture (Ch.10).
- SQL keywords are **not case-sensitive** (`SELECT = select`); string literal comparisons ARE case-sensitive per the SQL standard (though MySQL/SQL Server often ignore case by default).
- Default duplicate handling = **ALL** (keeps duplicates); use **DISTINCT** to remove them.

9.3 SELECT, WHERE, DISTINCT

- SELECT** = vertical filtering (choose columns) → maps to relational algebra **Projection (π)**.
- WHERE** = horizontal filtering (choose rows) → maps to relational algebra **Selection (σ)**.
- Comparison operators: `=`, `<`, `>`, `<=`, `>=`, `<>` (not equal). Logical connectives: `AND`, `OR`, `NOT`.
- BETWEEN low AND high** = shorthand for `>= low AND <= high` (inclusive both ends).
- DISTINCT** removes duplicate rows (based on the full selected column combination), not per-column.
- TRAP:** Arithmetic expressions in SELECT (e.g., `balance * 1.06`) only change **displayed** output — they never modify stored data.

9.4 SQL Data Types

Category	Types	Notes
Numeric	INT, DECIMAL(p,s), FLOAT	DECIMAL = exact (currency); FLOAT = approximate
String	CHAR(n), VARCHAR(n), TEXT	CHAR = fixed-length (padded); VARCHAR = variable-length; TEXT = long-form
Date/Time	DATE, DATETIME/TIME	DATE = YYYY-MM-DD only
Boolean	BOOLEAN	TRUE/FALSE

- TRAP:** **CHAR(n)** is **fixed-length** (pads with spaces to n) — **VARCHAR(n)** is **variable-length** (stores only actual content, up to n). CHAR wastes space for short values but is marginally faster for fixed-format data (e.g., state codes).

9.5 Operators, Precedence & NULL

- Categories: Arithmetic (`+`, `-`, `*`, `/`, `%`), Comparison (`=`, `!=`, `<`, `>`, `<=`, `>=`, `BETWEEN`, `LIKE`, `IN`, `IS NULL`), Logical (`AND`, `OR`, `NOT`), Set/Concatenation (`||`, `UNION`, `INTERSECT`, `EXCEPT`).
- NULL** = missing/unknown/not-applicable — **NOT the same as 0, empty string, or spaces**.
- TRAP:** `WHERE age = NULL` **always fails** (NULL isn't "equal" to anything, even itself) — you MUST use **IS NULL** / **IS NOT NULL** to test for it.

9.6 SQL Command Categories

Category	Full Name	Commands	Auto-commit?	Rollback?
DDL	Data Definition	CREATE, ALTER, DROP, TRUNCATE	Yes	No
DML	Data Manipulation	SELECT, INSERT, UPDATE, DELETE	No	Yes
DCL	Data Control	GRANT, REVOKE	Yes	No
TCL	Transaction Control	COMMIT, ROLLBACK, SAVEPOINT	—	—

DELETE vs TRUNCATE vs DROP (classic trap table)

Feature	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Action	Removes rows (supports WHERE)	Wipes ALL data, keeps structure	Removes table + structure entirely
Rollback?	Yes	No (auto-committed)	No
Fires triggers?	Yes	No	No
Speed	Slower (row-by-row + logging)	Faster (deallocates pages)	—

TRAP: This DELETE/TRUNCATE/DROP trio is one of the most frequently tested DBMS facts — remember DELETE is the only one of the three that is undoable (rollback-capable) and the only one that can be filtered with WHERE.

9.7 ALTER Command

```
ALTER TABLE Student ADD email VARCHAR(100);           -- add column
ALTER TABLE Student DROP COLUMN email;               -- remove column
ALTER TABLE Student MODIFY name VARCHAR(100);        -- change datatype/size
ALTER TABLE Student RENAME COLUMN marks TO score;    -- rename column
ALTER TABLE Student ADD CONSTRAINT pk_id PRIMARY KEY(id);
ALTER TABLE Student ADD CONSTRAINT fk_dept FOREIGN KEY(dept_id) REFERENCES Dept(id);
```

- **CASCADE** on a DROP removes the column/constraint AND everything depending on it (vs RESTRICT, which blocks the drop if dependencies exist).

9.8 DCL — GRANT / REVOKE

```
GRANT SELECT, UPDATE ON Student TO 'Teacher_User';
REVOKE UPDATE ON Student FROM 'Teacher_User';
```

- Common privileges: SELECT (read), INSERT, UPDATE, DELETE.

9.9 TCL — COMMIT / ROLLBACK / SAVEPOINT

- **COMMIT:** permanently saves all changes since the last COMMIT.
- **ROLLBACK:** undoes changes since the last COMMIT (or SAVEPOINT).
- **SAVEPOINT:** a named checkpoint within a transaction, so you can roll back partially rather than the whole transaction.

9.10 DESCRIBE / SHOW TABLES

Command	Shows
DESC / DESCRIBE <code>table_name</code>	Table blueprint: column names, types, NULL-ability, key info, defaults
SHOW TABLES (MySQL)	List of table names — Oracle instead uses <code>SELECT table_name FROM user_tables;</code>
SELECT * FROM <code>table</code>	The actual stored rows

9.11 ORDER BY & Aliases (Rename)

- **Alias (AS):** temporary name for a column/table, exists only for the query's duration — doesn't alter stored schema. Usable in both SELECT and FROM clauses.
- **ORDER BY:** sorts final result rows; default ascending (`ASC`), use `DESC` for descending.

Questions

Q1. [Trap] Which of DELETE, TRUNCATE, and DROP can be rolled back?

A. All three B. Only DELETE C. Only TRUNCATE D. None of them

Answer: B. **Why:** DELETE is a DML command (transaction-logged, rollback-capable); TRUNCATE and DROP are DDL and auto-commit immediately.

Q2. [Trap] `WHERE salary = NULL` in a SQL query will:

A. Return rows where salary is unknown B. Return rows where salary is exactly 0 C. Return no rows, because NULL cannot be compared with `=` D. Cause a syntax error

Answer: C. **Why:** NULL represents an unknown value; standard comparison operators always evaluate to unknown/false against NULL — `IS NULL` must be used instead.

Q3. What is the correct logical order of execution for `SELECT ... FROM ... WHERE ... ?`

A. SELECT → FROM → WHERE B. WHERE → FROM → SELECT C. FROM → WHERE → SELECT D. FROM → SELECT → WHERE

Answer: C. **Why:** The engine first identifies source tables (FROM), filters rows (WHERE), then picks columns to display (SELECT) — even though it's written SELECT-first.

Q4. A column declared as `CHAR(10)` storing the value `'Cat'` will actually occupy:

A. 3 characters, like VARCHAR B. 10 characters (padded with trailing spaces) C. 0 characters until updated D. An error, since 'Cat' is too short

Answer: B. **Why:** CHAR(n) is fixed-length — shorter values are space-padded to the full declared length, unlike VARCHAR(n) which stores only the actual content.

Q5. Which command is used to remove specific rows from a table using a WHERE clause, in a way that CAN be rolled back?

A. TRUNCATE B. DROP C. DELETE D. ALTER

Answer: C. **Why:** DELETE supports WHERE-based row filtering and is a DML command, so its effect is transaction-logged and reversible before COMMIT.

Q6. `SELECT DISTINCT name, city FROM Students;` will return:

A. Only unique names B. Only unique cities C. Unique (name, city) combinations, treating each pair as a unit D. All rows including duplicates

Answer: C. **Why:** DISTINCT with multiple columns removes duplicates based on the *combination* of all selected columns, not any single column alone.

Q7. Which SQL command category includes GRANT and REVOKE?

A. DDL B. DML C. DCL D. TCL

Answer: C. **Why:** DCL (Data Control Language) manages access permissions; GRANT gives privileges, REVOKE removes them.

Q8. [Trap] Which statement about DDL commands is correct?

A. DDL commands can always be rolled back like DML B. DDL commands (CREATE, ALTER, DROP, TRUNCATE) are auto-committed and generally cannot be rolled back C. DDL only affects data, never structure D. DDL requires a COMMIT statement to take effect

Answer: B. **Why:** DDL changes the schema and is auto-committed immediately — unlike DML, which stays reversible until COMMIT.

Q9. An alias created with `AS` in a query:

A. Permanently renames the column in the database B. Exists only for the duration of that single query C. Requires an ALTER TABLE statement D. Can only be used on tables, never columns

Answer: B. **Why:** Aliases are temporary display names scoped to the query — the underlying schema/column names remain unchanged.

Q10. A `DECIMAL(p,s)` data type is preferred over `FLOAT` for storing currency because:

A. DECIMAL uses less storage B. DECIMAL stores exact values, avoiding floating-point rounding errors C. FLOAT cannot store numbers at all D. DECIMAL is faster for all arithmetic

Answer: B. **Why:** DECIMAL(p,s) stores an exact fixed-point value (important for money), while FLOAT is an approximate binary representation prone to rounding errors.

10. SQL — Aggregates, Joins, Set Operations, Subqueries, Views

10.1 Aggregate Functions

- Five built-ins: **AVG, MIN, MAX, SUM, COUNT**.
- SUM / AVG require numeric input; MIN / MAX / COUNT also work on non-numeric (strings, dates).
- TRAP:** COUNT(*) counts ALL rows including NULLs; COUNT(column) skips NULL values in that column. All other aggregate functions (SUM, AVG, MIN, MAX) ignore NULLs automatically.
- AVG(balance) is NOT always equal to SUM(balance)/COUNT(balance) when NULLs are involved in a way that changes denominators — but when using COUNT(column) (which also skips NULLs) both forms match; COUNT(*) in the denominator would NOT match if NULLs exist (it'd divide by too many rows).

10.2 String Pattern Matching (LIKE) & Common String Functions

```
LIKE '%Comp%' -> matches any string containing "Comp" anywhere
LIKE '___' -> matches exactly 3 characters (each _ = exactly 1 char)
LIKE '___%' -> matches strings of AT LEAST 3 characters
NOT LIKE -> mismatches
ESCAPE 'char' -> lets % or _ be treated literally, e.g. LIKE 'ab\%cd%' ESCAPE '\'
```

Function	Purpose
UPPER / LOWER	Case conversion
LENGTH vs CHAR_LENGTH	LENGTH = bytes; CHAR_LENGTH = characters (differ for multi-byte chars)
CONCAT	Join strings
SUBSTRING(col, start, len)	Extract substring
REPLACE(col, old, new)	Replace text
TRIM / LTRIM / RTRIM	Remove spaces (both/left/right)
INSTR / POSITION	Find position of a substring

- TRAP:** LENGTH() measures **bytes**, CHAR_LENGTH() measures **characters** — identical for plain ASCII but diverge for multi-byte symbols (e.g., '©' → 1 character but 2 bytes).

10.3 Set Operations

```
UNION -> combines rows, REMOVES duplicates (like mathematical union)
UNION ALL -> combines rows, KEEPS duplicates
INTERSECT -> common rows only, removes duplicates (INTERSECT ALL keeps dupes)
EXCEPT/MINUS -> rows in first query but not second, removes duplicates (EXCEPT ALL keeps dupes)
```

- Requires **union-compatible** relations: same number of columns, matching/compatible domains.
- Cartesian Product:** a bare FROM r1, r2 (no join condition) = every tuple of r1 paired with every tuple of r2; commutative; size = |r1| × |r2|.

10.4 Joins — SQL Syntax

Join type	Behavior
INNER JOIN (or JOIN ... USING(col) / natural join)	Only matching rows from both tables
NATURAL JOIN	Auto-matches on ALL identically-named common columns; shows the common column only once
JOIN ... USING(col)	You explicitly pick which common column(s) must match — other same-named columns are ignored
LEFT OUTER JOIN	All left-table rows + matched right rows (NULL if no match)
RIGHT OUTER JOIN	All right-table rows + matched left rows (NULL if no match)
FULL OUTER JOIN	All rows from both sides, NULL-padded where unmatched

- TRAP:** Natural Join can silently misbehave if two tables share a common column name that ISN'T meant to be a join key — JOIN ... USING(col) avoids this by letting you pick exactly which column(s) to match on.
- Inner joins **lose unmatched tuples** from both sides — this "loss of information" is exactly what Outer Joins are designed to prevent (via NULL-padding).

10.5 GROUP BY & HAVING

Logical execution order with aggregation:
 FROM -> WHERE (filters ROWS, before grouping) -> GROUP BY (forms groups)
 -> HAVING (filters GROUPS, after aggregation) -> SELECT

- **Rule:** any SELECT column NOT in GROUP BY must appear **only inside an aggregate function** — otherwise the query is invalid (which row's value would it show for a group of many?).
- Same rule applies to HAVING: a non-aggregated attribute in HAVING must appear in GROUP BY.
- **TRAP: WHERE filters individual rows BEFORE grouping; HAVING filters entire GROUPS AFTER aggregation.** You cannot use an aggregate function (e.g., `AVG(balance)`) inside WHERE — it must go in HAVING, since the aggregate doesn't exist yet at the WHERE stage.

10.6 Subqueries — Simple vs Correlated

- **Simple (non-correlated) subquery:** inner query executes ONCE, independently, before the outer query uses its result.
- **Correlated subquery:** inner query REFERENCES a column from the outer query, so it must be **re-evaluated once per outer row** — much slower than a simple subquery.

```
-- Simple subquery: employees earning the max salary in the whole company
SELECT ename FROM Emp WHERE sal = (SELECT MAX(sal) FROM Emp);

-- Correlated subquery: employees earning above their OWN department's average
SELECT e.ename FROM Emp e
WHERE e.sal > (SELECT AVG(sal) FROM Emp WHERE dno = e.dno);
```

- **TRAP:** A correlated subquery cannot simply be "run first" — its result *depends on* the current outer row, so conceptually it re-runs per row (though real optimizers may rewrite it as a join for performance).

10.7 Views

- A **View** = a virtual table defined by a stored SELECT query; stores no data of its own, always reflects the underlying table(s) live.
- Uses: simplify complex/repeated queries, restrict access to sensitive columns/rows, present customized data.

```
CREATE VIEW student_result AS
SELECT name, marks FROM Student;
```

- **TRAP:** A view is generally **NOT updatable** (INSERT/UPDATE/DELETE through it will fail or be restricted) if its defining query contains: aggregate functions, GROUP BY, DISTINCT, or a JOIN across multiple base tables — because there's no unambiguous way to map the change back to base-table rows.

10.8 PL/SQL Basics (Oracle extension)

- PL/SQL = SQL + procedural constructs (loops, conditionals, variables) — used for procedures, functions, triggers.

```
DECLARE    -- optional: variable declarations
BEGIN      -- executable statements
EXCEPTION -- optional: error handling
END;
```

Component	Returns a value?	Called explicitly?
Procedure	No	Yes (EXEC name;)
Function	Yes	Yes (usable inside a SELECT)
Trigger	No	No — fires automatically on INSERT/UPDATE/DELETE

- **Trigger timing:** `BEFORE` (runs before the DML event) vs `AFTER` (runs after) INSERT/UPDATE/DELETE.

Questions

Q1. [Trap] For a `balance` column containing some NULLs, which pair of expressions always produce identical results?

A. `COUNT(*)` and `COUNT(balance)` B. `COUNT(balance)` and `SUM(balance)/AVG(balance)` C. `AVG(balance)` and `SUM(balance)/COUNT(balance)` D. `COUNT(*)` and `SUM(balance)`

Answer: C. **Why:** Both AVG and the manual SUM/COUNT(column) form skip NULL rows identically; COUNT(*) would count NULL rows too, making it diverge from COUNT(column).

Q2. [Trap] What is the key functional difference between WHERE and HAVING?

A. They are interchangeable B. WHERE filters rows before grouping; HAVING filters groups after aggregation C. HAVING is used only with SELECT * D. WHERE can use aggregate functions but HAVING cannot

Answer: B. **Why:** WHERE operates on raw rows prior to GROUP BY; HAVING operates on the aggregated groups afterward — hence only HAVING can filter on an aggregate condition like `AVG(balance) > 1500`.

Q3. A query pattern like `LIKE ' _ _ %'` (no spaces, 3 underscores then %) matches:

A. Exactly 3 characters B. At least 3 characters C. At most 3 characters D. Exactly 4 characters

Answer: B. **Why:** Each underscore matches exactly one character (3 mandatory chars), and `%` matches zero or more additional characters — so the total is "at least 3."

Q4. Which set operation keeps duplicate rows instead of removing them?

A. UNION B. INTERSECT C. UNION ALL D. EXCEPT

Answer: C. **Why:** Plain UNION/INTERSECT/EXCEPT all remove duplicates by default; appending ALL (e.g., UNION ALL) preserves duplicate rows.

Q5. [Trap] A correlated subquery is generally slower than a simple subquery because:

A. It uses more disk space B. It must conceptually re-evaluate once for every row of the outer query, since it references an outer column C. It cannot use WHERE clauses D. It always returns more rows

Answer: B. **Why:** Because a correlated subquery's result depends on the current outer row's value, it is (conceptually) re-executed per outer row, unlike an independent simple subquery executed once.

Q6. A view defined as `CREATE VIEW v AS SELECT dept, AVG(sal) FROM Emp GROUP BY dept;` is:

A. Fully updatable via INSERT/UPDATE B. Not updatable, because it contains an aggregate function and GROUP BY C. Updatable only for DELETE D. Identical to a base table

Answer: B. **Why:** Views containing aggregates/GROUP BY/DISTINCT/multi-table joins generally cannot be updated, since changes can't be unambiguously mapped back to base rows.

Q7. Which join type ensures NO row from either table is lost, padding unmatched sides with NULL?

A. INNER JOIN B. NATURAL JOIN C. FULL OUTER JOIN D. Cartesian Product

Answer: C. **Why:** FULL OUTER JOIN preserves unmatched rows from both the left and right relations, unlike inner/natural joins which only keep matching rows.

Q8. Which PL/SQL component MUST return a value and can be used directly inside a SELECT statement?

A. Procedure B. Trigger C. Function D. Block

Answer: C. **Why:** A Function always returns a value (e.g., `SELECT get_bonus FROM dual;`); a Procedure performs an action but returns nothing, and a Trigger fires automatically rather than being called explicitly.

Q9. [Numerical/Conceptual] `SELECT name FROM Emp WHERE name LIKE 'A%' AND name LIKE '%n';` returns names that:

A. Start with 'A' OR end with 'n' B. Start with 'A' AND end with 'n' C. Contain 'A' and 'n' anywhere D. Are exactly "An"

Answer: B. **Why:** The AND combines both LIKE conditions — the name must simultaneously start with 'A' and end with 'n' (e.g., "Arjun").

Q10. A trigger declared as `BEFORE INSERT ON Student` will:

A. Run after the row is inserted B. Run automatically before the INSERT completes, and must be called manually otherwise it won't fire C. Run automatically before the INSERT completes; it is never called explicitly D. Only run if explicitly invoked with EXEC

Answer: C. **Why:** Triggers fire automatically in response to the specified DML event (here, before an INSERT) — unlike procedures/functions, they are never called explicitly.

11. Tuple & Domain Relational Calculus

11.1 Declarative vs Procedural Query Languages

Language	Nature	Specifies
Relational Algebra (RA)	Procedural	HOW to get the result (explicit sequence of operations)
Tuple Relational Calculus (TRC)	Declarative	WHAT result is wanted (tuple-variable based)
Domain Relational Calculus (DRC)	Declarative	WHAT result is wanted (domain-variable based)

- SQL itself is declarative, and is formally grounded in both RA and TRC — this is why the "underlying model" question (RA=procedural, calculus=non-procedural) is a favorite exam fact.

11.2 Tuple Relational Calculus (TRC)

General form: $\{ t \mid P(t) \}$

- t is a **tuple variable** ranging over an entire relation; result = set of all tuples t for which predicate $P(t)$ is true.
- $t[A]$ or $t.A$ = value of tuple t on attribute A . $t \in r$ (or $r(t)$) = " t is a tuple of relation r ."
- Free vs Bound variable**: a tuple variable is **free** unless quantified by $\exists$ or $\forall$; quantified variables are **bound**.
- Quantifiers: $\exists t \in r (Q(t))$ = "there EXISTS a tuple t in r such that $Q(t)$ holds." $\forall t \in r (Q(t))$ = " $Q(t)$ holds for ALL tuples t in r ."
- Logical connectives allowed in $P(t)$: $\wedge$ (**AND**), $\vee$ (**OR**), $\neg$ (**NOT**), $\Rightarrow$ (**implies**).
- Key equivalences:

```

P1 AND P2      ≡ NOT(NOT(P1) OR NOT(P2))
FOR ALL t in r (P1(t)) ≡ NOT EXISTS t in r (NOT P1(t))    <- universal via negated existential
P1 => P2       ≡ NOT(P1) OR P2

```

Worked TRC Examples — Student(RollNo, Name, Branch)

Find all CSE students:

```
{t | Student(t) AND t.branch = 'CSE'}
```

Find Roll No only of CSE students (projection within calculus):

```
{t.RollNo | Student(t) AND t.branch = 'CSE'}
```

Worked TRC Example — banking schema (loan, borrower, depositor)

Loans over 1200:

```
{t | t ∈ loan AND t[amount] > 1200}
```

Customers with a loan AND an account (both):

```
{t | ∃ s ∈ borrower (t[cname]=s[cname]) AND ∃ u ∈ depositor (t[cname]=u[cname])}
```

Customers with a loan but NO account (existential + negation):

```
{t | ∃ s ∈ borrower (t[cname]=s[cname]) AND NOT ∃ u ∈ depositor (t[cname]=u[cname])}
```

- TRAP**: "Has X or Y" uses $\vee$ (**OR**) between two $\exists$ clauses; "has X but NOT Y" uses $\exists \dots$ **AND NOT** $\exists \dots$ — mixing these up is the single most common TRC mistake.

11.3 Domain Relational Calculus (DRC)

General form: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$

- Variables ($x_1 \dots x_n$) range over **individual domain values** (one variable per attribute), NOT whole tuples — this is the key structural difference from TRC.
- To produce a degree- n result relation, you need exactly n domain variables (one per output attribute).
- Atom forms: $\langle x_1, \dots, x_n \rangle \in r$ (membership), $x \text{ op } y$ (compare two domain variables), $x \text{ op } c$ (compare a domain variable to a constant).
- DRC is the theoretical basis of the **QBE (Query-By-Example)** language.

Worked DRC Examples

```

Student(RollNo, Name, Branch) – find CSE students' details:
{(RollNo,Name,Branch) | Student(RollNo,Name,Branch) AND Branch='CSE'}

Instructor(id,name,dept,salary) – salary > 80000:
{<i,n,d,s> | <i,n,d,s> ∈ Instructor AND s > 80000}

loan(loan_no,branch,amount) – branch/loan_no/amount for loans over 1200:
{<l,b,a> | <l,b,a> ∈ loan AND a > 1200}

```

11.4 Safety of Expressions

- A calculus expression must be **safe** — guaranteed to yield a **finite** result.
- Unsafe example:** $\{t \mid \text{NOT } (t \in r)\}$ — this could denote an infinite set (every tuple NOT in r , over an unbounded domain), which is meaningless for a real database.
- TRAP:** Negation and universal quantification ($\forall$) are the usual sources of unsafe expressions — always check that a negated/universal condition is still bounded by an explicit relation membership check.

11.5 RA vs TRC vs DRC — Summary

Feature	RA	TRC	DRC
Style	Procedural	Declarative	Declarative
Variable ranges over	(no variables — operator chain)	Whole tuples	Single domain values
Basis for	—	Part of SQL's theoretical foundation	QBE language
"For all" queries	Division operator	$\forall$ quantifier	$\forall$ quantifier

Questions

Q1. Which of the following best distinguishes Relational Algebra from Relational Calculus?

A. RA is declarative, Calculus is procedural B. RA is procedural (specifies how), Calculus is declarative (specifies what) C. They are functionally identical with no difference D. Calculus cannot express joins

Answer: B. **Why:** RA is a step-by-step procedural language; both TRC and DRC are declarative, specifying only the desired result condition.

Q2. In TRC, a tuple variable that is NOT quantified by $\exists$ or $\forall$ is called:

A. Bound B. Free C. Universal D. Undefined

Answer: B. **Why:** By definition, a tuple variable is free unless explicitly quantified (bound) by an existential or universal quantifier.

Q3. [Trap] Which TRC expression correctly finds customers who have a loan but NO account?

A. $\{t \mid \exists s \in \text{borrower}(t[\text{cname}]=s[\text{cname}]) \text{ OR } \exists u \in \text{depositor}(t[\text{cname}]=u[\text{cname}])\}$ B. $\{t \mid \exists s \in \text{borrower}(t[\text{cname}]=s[\text{cname}]) \text{ AND } \exists u \in \text{depositor}(t[\text{cname}]=u[\text{cname}])\}$ C. $\{t \mid \exists s \in \text{borrower}(t[\text{cname}]=s[\text{cname}]) \text{ AND NOT } \exists u \in \text{depositor}(t[\text{cname}]=u[\text{cname}])\}$ D. $\{t \mid \text{NOT } \exists s \in \text{borrower}(t[\text{cname}]=s[\text{cname}])\}$

Answer: C. **Why:** "Has a loan" needs an existential match in borrower; "but no account" needs the negation of an existential match in depositor — combined with AND.

Q4. In Domain Relational Calculus, a query producing a degree-3 result relation requires:

A. 1 domain variable B. Exactly 3 domain variables (one per output attribute) C. As many variables as base relations used D. No variables, only constants

Answer: B. **Why:** DRC requires one domain variable per output attribute — a degree-3 result needs 3 domain variables, e.g., $\langle x1, x2, x3 \rangle$.

Q5. [Trap] Why is $\{t \mid \text{NOT}(t \in r)\}$ considered an unsafe expression?

A. It uses invalid syntax B. It could denote an infinite set of tuples, since $\text{NOT}(t \in r)$ is unbounded over the whole domain C. NOT is not allowed in relational calculus at all D. It only works for empty relations

Answer: B. **Why:** Without a bounding relation membership, the negation could range over an infinite domain of possible tuples — violating safety (finiteness) requirements.

Q6. The universal quantifier $\forall$ in relational calculus is most analogous to which relational algebra operator?

A. Selection (σ) B. Projection (π) C. Division ($\div$) D. Cartesian Product ($\times$)

Answer: C. **Why:** Both $\forall$ ("for all") and Division express "related to ALL values of some set" — e.g., "find X related to every Y."

Q7. Domain Relational Calculus is described as the theoretical basis for which practical query language?

A. SQL B. QBE (Query-By-Example) C. Relational Algebra D. PL/SQL

Answer: B. **Why:** DRC's domain-variable-per-attribute structure directly underlies QBE's grid-based query interface.

12. Transactions, ACID Properties & Serializability

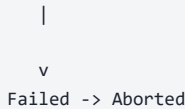
(Serializability testing via precedence graph is one of the most numerically-tested DBMS topics in GATE/ISRO.)

12.1 Transaction Basics & ACID

- **Transaction** = a set of logically related instructions forming one logical unit of work; either executes fully or not at all (removes the "partial execution" problem that plain OS-level instruction atomicity can't prevent).
- Two DB-level operations: **Read(X)** (disk → memory), **Write(X)** (memory → disk).
- **ACID**:
- **Atomicity** — all-or-nothing; enforced by the recovery manager.
- **Consistency** — DB moves from one consistent state to another; enforced by programmers/integrity constraints.
- **Isolation** — concurrent transactions don't interfere, as if run in isolation; enforced by the concurrency control manager.
- **Durability** — committed changes survive failures; enforced by the recovery manager.
- **TRAP**: Isolation is the concurrency-control manager's job; Atomicity & Durability are the recovery manager's job; Consistency is the programmer's/constraint's job — a commonly confused mapping.

12.2 Transaction States

Active → Partially Committed → Committed



- **Active**: executing. **Partially committed**: final statement executed, but output may still be in memory (not yet safely on disk) — can still be forced to abort. **Failed**: cannot proceed further (must roll back). **Aborted**: rolled back, DB restored to pre-transaction state. **Committed**: successfully finished, changes durable.

12.3 Why Concurrency — and Its Anomalies

- Concurrent execution improves throughput, resource utilization, and response/waiting time — but naive interleaving can violate consistency even if each transaction individually respects ACID.

Anomaly	What happens
Lost Update (write-write)	Two transactions write the same item with no read in between; the 2nd write silently overwrites the 1st.
Dirty Read (read-write)	A transaction reads a value written by an uncommitted transaction, which later aborts — the reader now holds a value that never really existed.
Unrepeatable Read	A transaction reads the same item twice and gets different values because another transaction updated it in between.
Phantom Read	A transaction re-executes a query and finds new/vanished rows because another transaction inserted/deleted matching rows in between.

- **TRAP**: Dirty Read is about reading an **uncommitted** write; Unrepeatable Read is about a value **changing** between two reads of the same item (both transactions may be committed) — don't conflate the two.

12.4 Schedules

- **Schedule**: an interleaving of operations from multiple transactions; each transaction's own operations must keep their original relative order.
- **Serial schedule**: one transaction runs fully, then the next (no interleaving). For n transactions, there are $n!$ serial schedules. Every serial schedule is trivially consistent, but throughput is poor.
- **Non-serial schedule**: operations interleaved across transactions (real concurrent execution).

Total possible schedules (given instruction counts $n_1, n_2, \dots, n_k$ for k transactions)
 $= (n_1 + n_2 + \dots + n_k)! / (n_1! \times n_2! \times \dots \times n_k!)$
 Non-serial schedule count = Total - $k!$ ($k!$ = the serial ones)

Worked Example

T1 has 2 instructions, T2 has 3 instructions.
 Total schedules = $(2+3)! / (2! \times 3!) = 120/12 = 10$.
 Serial schedules = $2! = 2$ (T1-then-T2, or T2-then-T1).
 Non-serial schedules = $10 - 2 = 8$.

12.5 Conflict Serializability

- Two operations **conflict** iff: (1) different transactions, (2) same data item, (3) at least one is a WRITE.

Read-Read → Never conflicts
 Read-Write / Write-Read / Write-Write → Conflicts (if different transactions, same item)

- Conflict equivalent:** schedule S can be transformed into S' purely by swapping adjacent *non-conflicting* operations.
- Conflict serializable:** S is conflict-equivalent to some serial schedule.

Precedence Graph Method (the standard test)

- One node per transaction.
- For every pair of conflicting operations, add edge $T_i \rightarrow T_j$ if T_i 's operation occurs BEFORE T_j 's operation in S.
- Schedule is conflict serializable $\Leftrightarrow$ the graph has NO CYCLE.
- If acyclic, any topological order of the graph is a valid equivalent serial schedule.

- TRAP:** You must check EVERY conflicting pair across the whole schedule, not just "adjacent" operations — a single missed conflicting pair can hide a cycle.

Worked Example

S: R1(A) R2(B) W1(A) R3(A) W2(B) W3(A)

Conflicts on A: R1(A)→W3(A) gives **T1→T3**; W1(A)→R3(A) gives **T1→T3** (same edge, no new info); R1(A) vs R3(A) don't conflict (both reads).

Conflicts on B: only T2 touches B (R2(B), W2(B)) → same transaction, no edge.

Graph edges: **T1→T3** only. T2 is unconstrained.

Acyclic ⇒ conflict serializable. Valid equivalent serial orders: **T1,T2,T3** or **T2,T1,T3** or **T1,T3,T2** (any order respecting T1 before T3).

12.6 View Serializability (weaker/broader than conflict serializability)

- View equivalent** (S and S'): (1) same "reads-from" relationships — if T_i reads the initial value of Q in S, it must also read the initial value of Q in S'; if T_i reads Q as written by T_j in S, same in S'; (2) same **final writer** of each data item in both schedules.
- View serializable:** S is view-equivalent to some serial schedule. Checking this is NP-complete in general (compare against all $n!$ serial orders).
- Key containment fact:** every **conflict serializable** schedule is **view serializable**, but NOT every view serializable schedule is conflict serializable.
- TRAP:** Direction matters — "conflict serializable ⇒ view serializable" is TRUE; the reverse ("view serializable ⇒ conflict serializable") is FALSE. A classic exam trap reverses this.
- When they can differ:** only when the schedule has a **blind write** (a write with no prior read of that item by the same transaction, in that schedule). With no blind writes, "not conflict serializable" ⇒ "not view serializable" too.

Worked Example (the classic conflict-vs-view-serializable gap)

S: R1(B) R3(C) R1(A) W2(A) W1(A) W2(B) W3(A) W1(B) W3(B) W3(C)

Precedence graph: on A → $T_1 \rightarrow T_2$ (R1(A) before W2(A)) **and** $T_2 \rightarrow T_1$ (W2(A) before W1(A)) — **a 2-cycle ⇒ NOT conflict serializable**.

But T2's writes W2(A), W2(B) are **blind writes** (T2 never reads). Checking view-equivalence: T1 reads initial A and B (so T1 must precede any writer of A/B in a serial order); T3 is the final writer of A, B, C in S (so T3 must be last). Checking **T1, T2, T3**: reads-from and final-writer conditions both match S exactly.

⇒ S is **view serializable** (equivalent to serial T1,T2,T3) **despite not being conflict serializable** — precisely because the "conflicting" writes by T2 were blind and never actually observed by anyone.

Questions

Q1. [Numerical] T1 has 3 instructions and T2 has 2 instructions. How many non-serial schedules are possible?

A. 10 B. 8 C. 120 D. 6

Answer: B. **Why:** Total = $5!/(3!2!) = 10$; serial = $2! = 2$; non-serial = $10 - 2 = 8$.

Q2. Two operations belonging to different transactions on the same data item, where both are READs, are:

A. Always conflicting B. Never conflicting C. Conflicting only if isolation level is low D. Conflicting only in strict schedules

Answer: B. **Why:** Conflict requires at least one WRITE; two reads never conflict regardless of transaction or isolation level.

Q3. [Application] Schedule S: R1(x) R1(y) R2(x) R2(y) W2(y) W1(x) — is S conflict serializable?

A. Yes, equivalent to T1,T2 B. Yes, equivalent to T2,T1 C. No — the precedence graph has a cycle D. Cannot be determined

Answer: C. **Why:** On x: R1(x)@1 before R2(x)@3 (no conflict, both reads only relevant if later write — actually the conflict is R2(x) then W1(x): T2→T1). On y: R1(y)@2 then W2(y)@5 gives T1→T2. Combined edges T1→T2 (from y) and T2→T1 (from x) form a 2-cycle ⇒ not conflict serializable.

Q4. [Trap] Which statement about conflict and view serializability is TRUE?

A. Every view serializable schedule is conflict serializable B. Every conflict serializable schedule is view serializable C. The two are always identical D. Neither implies the other

Answer: B. **Why:** Conflict serializability is the stronger, narrower condition; it always implies view serializability. The reverse implication fails whenever blind writes are present.

Q5. A schedule has a cycle in its precedence graph but contains blind writes. It is:

A. Definitely not view serializable either B. Possibly still view serializable — must check view-equivalence conditions directly C. Automatically view serializable D. Automatically strict

Answer: B. **Why:** A precedence-graph cycle only rules out conflict serializability. With blind writes present, the schedule might still satisfy the (weaker) view-equivalence conditions — it must be checked separately, not assumed either way.

Q6. In a serial schedule of n transactions, how many distinct serial schedules exist regardless of each transaction's instruction count?

A. n B. $n!$ C. 2^n D. n^2

Answer: B. **Why:** A serial schedule is just an ordering of the n transactions as whole blocks — $n!$ such orderings exist, independent of how many instructions each transaction has.

Q7. [Trap] A transaction T_2 reads a value written by transaction T_1 , which later ABORTS. This is an example of:

A. Lost update B. Dirty read C. Unrepeatable read D. Phantom read

Answer: B. **Why:** Reading data written by an uncommitted (and here, ultimately aborted) transaction is precisely the dirty-read anomaly.

Q8. [Application] Schedule S : $R_2(x) W_2(x) R_2(y) W_2(y) R_1(y) W_1(x)$ — checking conflicts between T_1 and T_2 only, which edge appears in the precedence graph?

A. $T_1 \rightarrow T_2$ only B. $T_2 \rightarrow T_1$ only C. Both $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$ (cycle) D. No edge at all

Answer: B. **Why:** On y : $W_2(y)$ precedes $R_1(y) \Rightarrow T_2 \rightarrow T_1$. On x : $W_2(x)$ precedes $W_1(x) \Rightarrow T_2 \rightarrow T_1$. Both conflicts point the same direction ($T_2 \rightarrow T_1$), so no cycle — this schedule is conflict serializable (equivalent to T_2, T_1).

Q9. Which of the following is/are TRUE?

A. All view serializable schedules are also conflict serializable B. All conflict serializable schedules are also view serializable, and if a schedule is not view serializable then it is not conflict serializable C. Conflict and view serializability are unrelated concepts D. View serializability is always easier to compute than conflict serializability

Answer: B. **Why:** "Conflict serializable $\Rightarrow$ view serializable" and its logical contrapositive "not view serializable $\Rightarrow$ not conflict serializable" are both true; the converse statements (A) are false, and checking view serializability is NP-complete — harder, not easier, than the polynomial-time precedence-graph check for conflict serializability.

13. Recoverability & Concurrency Control Protocols

13.1 Recoverability Classification

- **Recoverable schedule:** if T_j reads a data item previously written by T_i , then T_i must **commit before** T_j commits.
- **Cascadeless schedule:** if T_j reads a data item previously written by T_i , then T_i must **commit before** T_j even **reads** it (i.e., no dirty reads at all). Avoids **cascading rollback** (one abort forcing a chain of other rollbacks).
- **Strict schedule:** if T_j reads OR writes a data item previously written by T_i , T_i must commit before T_j 's read/write. Strongest — allows trivial UNDO via before-images since no one ever touched an uncommitted value.

Containment hierarchy (strict nesting):

Strict $\subset$ Cascadeless $\subset$ Recoverable $\subset$ All Schedules

- **TRAP:** This is a strict one-way nesting — every Strict schedule is Cascadeless, every Cascadeless schedule is Recoverable, but NOT vice versa. A schedule can be Recoverable without being Cascadeless, and Cascadeless without being Strict.

Quick Classification Procedure

1. Scan for a DIRTY READ (T_i reads an item written by an uncommitted T_j).
2. No dirty read anywhere $\Rightarrow$ schedule is BOTH Recoverable AND Cascadeless.
3. Dirty read present $\Rightarrow$ NOT Cascadeless. Then check commit order:
 - If every writer commits before its dirty-reader commits $\Rightarrow$ Recoverable (but not cascadeless).
 - If any dirty-reader commits before its writer commits $\Rightarrow$ NOT Recoverable.

Worked Examples (verified)

- A) $W_1(X); R_2(X); C_1; C_2;$ $\rightarrow$ T_2 dirty-reads X from uncommitted T_1 (not cascadeless), but T_1 commits before T_2 commits $\rightarrow$ RECOVERABLE, not cascadeless.
- B) $W_1(X); W_2(X); C_1; C_2;$ $\rightarrow$ T_2 overwrites X before T_1 commits (allowed - cascadeless only restricts READS of uncommitted data, not writes); nobody reads X $\rightarrow$ RECOVERABLE and CASCADELESS, but NOT STRICT (a write touched X before T_1 's commit).
- C) $W_1(X); C_1; W_2(X); C_2;$ $\rightarrow$ T_2 only touches X after T_1 already committed $\rightarrow$ STRICT (and therefore also cascadeless and recoverable).

- **TRAP:** Cascadelessness only cares about dirty **reads**; a transaction is free to **write** over another's uncommitted data and still be cascadeless. Only **strictness** also blocks that early write.

13.2 Concurrency Control — Overview

- Goal: guarantee (conflict) serializability while maximizing concurrency, with acceptable overhead.
- Three main approaches: **Lock-based** (2PL family), **Timestamp-ordering**, **Validation/optimistic** (for mostly-read workloads with low conflict rates).

13.3 Timestamp Ordering Protocol (TO)

- Each transaction gets a unique, fixed **TS(Ti)** on entry (never repeats, no reset needed); serialization order = timestamp order.
- Each data item Q tracks **W-timestamp(Q)** (latest committed-write's TS) and **R-timestamp(Q)** (latest successful read's TS).

Read(Q) by T_i :

```
if TS(Ti) < W-timestamp(Q): REJECT, roll back Ti  (Ti would read an already-overwritten value)
else: allow; R-timestamp(Q) = max(R-timestamp(Q), TS(Ti))
```

Write(Q) by T_i :

```
if TS(Ti) < R-timestamp(Q): REJECT, roll back Ti  (a later txn already needed a newer value)
elif TS(Ti) < W-timestamp(Q): REJECT, roll back Ti  (Ti's write is already obsolete)
else: allow; W-timestamp(Q) = max(W-timestamp(Q), TS(Ti))
```

- A rolled-back transaction is restarted with a **new, later** timestamp.
- **TRAP:** TO guarantees conflict serializability (conflicting ops are always processed in TS order, so the precedence graph can never cycle) and hence view serializability too — but it does **NOT** guarantee recoverability or cascadelessness (no rule ties commit order to anything), and it IS deadlock-free (a transaction is immediately rejected/rolled back rather than made to wait).

13.4 Thomas's Write Rule (TO optimization)

- Modifies ONLY the write rule: if $TS(T_i) < W\text{-timestamp}(Q)$, instead of rejecting, **simply ignore** that write (it's an obsolete write that no future reader will ever see, so skipping it is harmless).

- **TRAP:** The **read rule is completely unchanged** from basic TO — Thomas's rule improves concurrency for **writes only**. A read that fails the TO read-check ($TS(T_i) < W\text{-timestamp}(Q)$) is still rejected exactly as in basic TO.
- Net effect: allows some **view-serializable** (but not conflict-serializable) schedules that basic TO would have rejected — trades strict conflict-serializability for higher concurrency.

13.5 Lock-Based Protocols & Two-Phase Locking (2PL)

- **Shared (S) lock:** read-only, multiple transactions may hold S on the same item simultaneously. **Exclusive (X) lock:** read+write, no other transaction may hold any lock on that item. Compatibility: S-S = compatible; S-X, X-X = incompatible.
- Plain locking (lock/unlock freely) does NOT guarantee serializability by itself — need a locking discipline.
- **Two-Phase Locking (2PL):** every transaction (not the whole schedule) splits into a **Growing phase** (only acquire locks, never release) and a **Shrinking phase** (only release locks, never acquire). Once the first unlock happens, no more locks may be taken.
- **2PL guarantees conflict serializability** (and hence view serializability) — the equivalent serial order follows the transactions' **lock-point** order. It does **NOT** guarantee deadlock-freedom, recoverability, or cascadelessness by itself.
- **TRAP:** "If a schedule follows 2PL, it is conflict serializable" is TRUE, but the converse is FALSE — a schedule can be conflict serializable without ever being produced by 2PL.

13.6 2PL Variants — Complete Comparison

Conservative 2PL: acquire ALL needed locks BEFORE starting execution (or wait entirely) - no true growing phase overlap with execution. Needs advance knowledge of data needs.
 Basic 2PL: standard growing/shrinking split, locks released as soon as no longer needed.
 Rigorous 2PL: hold ALL locks (both S and X) until commit - no real shrinking phase at all.
 Strict 2PL: hold only X-locks until commit (S-locks may be released earlier) - simplified Rigorous 2PL; most widely used in real DBMSs.

Protocol	Conflict Serializable	View Serializable	Recoverable	Cascadeless	Deadlock-free
Timestamp Ordering	YES	YES	NO	NO	YES
Thomas's Write Rule	NO	YES	NO	NO	YES
Basic 2PL	YES	YES	NO	NO	NO
Conservative 2PL	YES	YES	NO	NO	YES
Rigorous 2PL	YES	YES	YES	YES	NO
Strict 2PL	YES	YES	YES	YES	NO

- **TRAP:** Conservative 2PL is the ONLY 2PL variant that is deadlock-free (since it never lets a transaction hold some locks while waiting for others — no hold-and-wait). Rigorous/Strict 2PL give recoverability+cascadelessness (via strictness) but still risk deadlock.
- **TRAP:** Thomas's Write Rule is the only protocol in this table that sacrifices conflict serializability (NO) while keeping view serializability (YES) — a direct consequence of it deliberately allowing certain blind-write reorderings.

Questions

Q1. [Application] In schedule $W1(X); R2(X); C1; C2$; — classify it.

A. Not recoverable B. Recoverable but not cascadeless C. Cascadeless but not recoverable D. Strict

Answer: B. **Why:** T2 dirty-reads X from uncommitted T1 (violates cascadelessness), but T1 commits before T2 commits, satisfying recoverability.

Q2. [Trap] Which is the correct containment relationship among schedule classes?

A. Recoverable $\subset$ Cascadeless $\subset$ Strict B. Strict $\subset$ Cascadeless $\subset$ Recoverable C. All three are equivalent D. Cascadeless $\subset$ Strict $\subset$ Recoverable

Answer: B. **Why:** Strict is the most restrictive (implies Cascadeless), Cascadeless is more restrictive than Recoverable — the nesting narrows as you add restrictions, so Strict is the smallest, most-restrictive set.

Q3. Under the Timestamp Ordering protocol, a Read(Q) by T_i is rejected when:

A. $TS(T_i) < R\text{-timestamp}(Q)$ B. $TS(T_i) < W\text{-timestamp}(Q)$ C. $TS(T_i) > W\text{-timestamp}(Q)$ D. $TS(T_i) = W\text{-timestamp}(Q)$

Answer: B. **Why:** If T_i 's timestamp is older than the latest committed write's timestamp, T_i would be reading an already-overwritten (obsolete) value, so it is rejected and rolled back.

Q4. [Trap] Thomas's Write Rule improves on basic Timestamp Ordering by:

A. Relaxing both the read and write rejection rules B. Relaxing ONLY the write rule (ignoring obsolete blind writes instead of rejecting them); the read rule is unchanged C. Eliminating rollbacks entirely D. Making the protocol deadlock-prone

Answer: B. **Why:** Thomas's rule specifically allows an obsolete write to be silently ignored rather than triggering a rollback — but reads still follow the exact same rejection rule as basic TO.

Q5. Which 2PL variant is guaranteed to be deadlock-free?

A. Basic 2PL B. Strict 2PL C. Conservative 2PL D. Rigorous 2PL

Answer: C. **Why:** Conservative 2PL acquires all required locks before execution begins (or waits entirely), eliminating the hold-and-wait condition necessary for deadlock.

Q6. [Trap] Basic 2PL guarantees conflict serializability. Does it also guarantee recoverability?

A. Yes, always B. No — a schedule can follow 2PL and still not be recoverable C. Only if all locks are exclusive D. Only for read-only transactions

Answer: B. Why: 2PL constrains only lock acquisition/release ordering relative to conflicts; it says nothing about commit ordering, so a non-recoverable schedule can still legally arise under Basic 2PL.

Q7. The key structural difference between Strict 2PL and Rigorous 2PL is:

A. Strict 2PL holds only X-locks until commit; Rigorous 2PL holds both S and X locks until commit B. Rigorous 2PL has no growing phase C. Strict 2PL is not two-phase D. They are identical in every respect

Answer: A. Why: Rigorous 2PL is the more conservative variant, retaining shared locks too until commit, whereas Strict 2PL allows shared locks to be released earlier — both still guarantee recoverability/cascadelessness via strictness on writes.

Q8. [Application] Which protocol allows a schedule to be view serializable but explicitly NOT conflict serializable?

A. Strict 2PL B. Conservative 2PL C. Thomas's Write Rule D. Basic Timestamp Ordering

Answer: C. Why: Thomas's Write Rule intentionally permits ignoring certain obsolete (blind) writes, which can produce schedules that are view serializable without being conflict serializable — the only protocol in this set with that property.

Q9. A schedule with no dirty reads anywhere is automatically:

A. Only recoverable, not cascadeless B. Both recoverable and cascadeless C. Neither, more checks needed D. Automatically strict as well

Answer: B. Why: With zero dirty reads, both the cascadeless condition (no reads of uncommitted data) and recoverability (trivially, since no problematic reads-from relationships exist) are automatically satisfied. Strictness is a separate, stronger condition about writes too and isn't automatically implied.

14. Database Recovery

14.1 Storage Types & Buffer Management Policies

- **Volatile storage** (RAM/cache) — lost on crash/power-off. **Non-volatile storage** (disk, tape, SSD) — survives. **Stable storage** — an idealized concept (survives ALL failures); approximated via multiple copies on different non-volatile devices.
- Buffer management policy pairs (determine WHEN dirty pages reach disk):
- **Steal** — an uncommitted transaction's page CAN be written to disk before commit. **No-steal** — cannot.
- **Force** — all pages of a transaction are flushed to disk immediately at commit. **No-force** — they aren't (flushed later, lazily).
- **TRAP: Steal + No-force** is what real systems use (best performance) — but this combination is exactly why **both UNDO and REDO are needed** during recovery (steal → uncommitted work may be on disk needing undo; no-force → committed work may NOT be on disk needing redo).

14.2 Log-Based Recovery & WAL

- **Log record types:** `<Ti start>`, `<Ti, Xj, V1, V2>` (update: old value V1, new value V2), `<Ti commit>`, `<Ti abort>`.
- **Write-Ahead Logging (WAL) — the single most important rule in this chapter:**

Atomicity Rule: the log entry for an insert/update/delete MUST reach stable storage
BEFORE the actual data change is written to the database on disk.
Durability Rule: ALL log entries for a transaction must reach disk
BEFORE that transaction's commit record is written to disk.

- **TRAP:** WAL means **log-first, data-second** — students often reverse this. Without WAL, a crash between the data-write and log-write would leave an update with no way to undo it.
- Each log record gets a unique **LSN (Log Sequence Number)**; **undo** restores the old value, **redo** re-applies the new value — both operations must be **idempotent** (repeating them has the same effect as doing them once, since recovery may re-scan the same log records).

14.3 Deferred vs Immediate Update

Feature	Deferred Update (No-Undo/Redo)	Immediate Update (Undo/Redo)
DB updated on disk...	...only AFTER commit	...possibly BEFORE commit
On crash, need UNDO?	Never (DB never touched by uncommitted txn)	Yes , for uncommitted transactions
On crash, need REDO?	Yes, for committed transactions	Yes, for committed transactions
Redo condition	Both <code><Ti start></code> and <code><Ti commit></code> present in log	<code><Ti commit></code> present (redo); absent (undo)

- **TRAP:** "No-Undo/Redo" (Deferred) is named for what recovery must do, NOT for what happens during normal execution — a transaction still executes normally, it just doesn't touch the actual DB on disk until commit.

14.4 Checkpoints

- Periodically: (1) flush all in-memory log records to stable storage, (2) flush all modified buffer pages to disk, (3) write a `<checkpoint L>` log record (L = list of transactions active at checkpoint time). Updates are paused during this process.
- **Purpose:** bounds how far back the recovery log must be scanned — transactions that committed *before* the checkpoint need no redo, cutting recovery time drastically on a long-running log.

14.5 Shadow Paging

- Alternative to logging: maintain **current page table** + **shadow page table** (identical at start). First write to any page → copy-on-write to a new page; current table repoints to the copy; shadow table is untouched.
- **Commit** = atomically flip a single on-disk pointer so the current table becomes the new shadow table.
- Advantages: **no log needed, no undo/redo, trivial/instant recovery** (crash before commit → old shadow table is still valid, nothing to undo).
- Disadvantages: data fragmentation (related pages scattered), needs garbage collection of orphaned pages, expensive to copy the whole page table each transaction (mitigated with a B+-tree-structured page table so only the path to the updated leaf is copied), poor support for concurrent transactions. Rarely used in modern practice — **TRAP:** don't assume shadow paging is "the standard" method; log-based (WAL/ARIES) recovery is what real systems use.

14.6 ARIES Recovery Algorithm

(ARIES = *Algorithms for Recovery and Isolation Exploiting Semantics* — the industry-standard log-based recovery algorithm.)

3 passes, always in this order:

1. ANALYSIS — scan FORWARD from the last checkpoint.
Rebuilds: Transaction Table (active txns) + Dirty Page Table (pages modified but maybe not on disk).
Determines: RedoLSN = where the Redo pass must start.
2. REDO — scan FORWARD from RedoLSN.
Re-applies EVERY logged update from that point on -- even for transactions that will later be undone ("repeating history"). Uses PageLSN vs log-record LSN to skip redos already reflected on disk (idempotence).
3. UNDO — scan BACKWARD from the end of the log.
Rolls back all "loser" transactions (active/uncommitted at crash time), writing a Compensation Log Record (CLR) for each undone action so undo is never repeated.

- Key data structures: **LSN** (sequentially increasing log position), **PageLSN** (LSN of the last update reflected on that page — prevents redoing an already-applied update), **Dirty Page Table** (RecLSN = earliest log record that dirtied this page — lets Redo skip pages/records that are provably already safe on disk), **CLR** (redo-only log record written during undo, so undo actions are never re-undone).
- **TRAP:** ARIES deliberately **redoes even the actions of transactions that will be undone next** ("repeating history") — this looks wasteful but massively simplifies the algorithm (Redo never needs to check whether a transaction eventually committed; Undo cleans up afterward).
- **TRAP:** The pass order is always **Analysis** → **Redo** → **Undo**, never Undo-before-Redo.

Questions

Q1. What is the correct order of ARIES recovery phases?

A. Redo, Undo, Analysis B. Analysis, Undo, Redo C. Analysis, Redo, Undo D. Undo, Analysis, Redo

Answer: C. **Why:** Analysis first rebuilds the transaction/dirty-page tables and finds where Redo should start; Redo re-applies all logged updates; Undo then rolls back uncommitted ("loser") transactions.

Q2. [Trap] The Write-Ahead Logging rule requires that:

A. Data is written to disk before its log record B. The log record for a change must reach stable storage before the data change itself is written to disk C. Only committed transactions are logged D. Checkpoints replace the need for logging

Answer: B. **Why:** WAL guarantees the log always has enough information to undo/redo a change, which only works if the log entry is durable before the actual data page is overwritten.

Q3. Under the Deferred Update (No-Undo/Redo) technique, why is UNDO never required during recovery?

A. Because logging is disabled B. Because the database on disk is never modified by a transaction until after it commits C. Because checkpoints eliminate the need D. Because only redo-only CLRs are used

Answer: B. **Why:** Since updates are deferred and only written to the DB after commit, an uncommitted transaction can never have left any trace on disk to undo.

Q4. A transaction's log shows `<Ti start>` but no `<Ti commit>` at the time of a crash under the Immediate Update scheme. What recovery action is needed?

A. Redo Ti B. Undo Ti C. No action needed D. Redo then Undo Ti

Answer: B. **Why:** Immediate update may have written some of Ti's changes to disk before the crash; since Ti never committed, those changes must be undone using the log's old values.

Q5. What is the main advantage of Shadow Paging over log-based recovery?

A. Better concurrency support B. No log records are needed; recovery is nearly instantaneous since commit is a single atomic pointer switch C. It uses less disk space than logging overall D. It avoids garbage collection

Answer: B. **Why:** Shadow paging's commit is just repointing to the current page table, so a crash before commit leaves the old shadow table fully valid — no undo/redo machinery needed. Its downsides are fragmentation, garbage collection, and poor concurrency.

Q6. [Trap] Why does ARIES redo the actions of transactions that will later be undone?

A. It is a bug in the algorithm B. "Repeating history" simplifies the Redo pass by not requiring it to know a transaction's eventual fate; Undo cleans up afterward C. Because Undo cannot run without a prior Redo pass touching those pages D. Because CLRs require it

Answer: B. **Why:** ARIES intentionally decouples Redo from transaction outcome — Redo restores the exact pre-crash state, and Undo then removes the effects of any transaction that hadn't committed.

Q7. What information does ARIES's PageLSN serve to prevent?

A. Duplicate log entries B. Redoing an update that is already reflected in the on-disk page C. Undoing committed transactions D. Checkpoint corruption

Answer: B. **Why:** By comparing a log record's LSN to the page's stored PageLSN, the Redo pass can skip updates already durably applied — ensuring idempotent recovery.

Q8. In the Steal/No-Force buffer policy combination used by most real DBMSs, which recovery operations become necessary?

A. Neither undo nor redo B. Only redo C. Only undo D. Both undo and redo

Answer: D. **Why:** "Steal" means uncommitted data can reach disk (needs undo on crash); "No-force" means committed data might NOT yet be on disk (needs redo on crash) — together they require both.

Q9. A Compensation Log Record (CLR) written during the ARIES Undo pass is best described as:

A. A record that itself will need to be undone later B. A redo-only record logging an undo action, which is never undone again C. A duplicate of the original update record D. Only used during the Analysis pass

Answer: B. **Why:** CLRs record the undo action taken during recovery; they are redo-only, ensuring that if a second crash occurs mid-recovery, the already-completed undo isn't repeated.

15. Data Warehousing, Mining, Distributed DB & NoSQL

15.1 Data Warehouse — Definition & Architecture

- **W.H. Inmon's definition (classic exam recall):** a data warehouse is **Subject-Oriented, Integrated, Time-Variant, and Non-Volatile** — a collection of data supporting management decision-making.
- **Subject-oriented:** organized around subjects (customer, sales), not daily transactions.
- **Integrated:** merges multiple heterogeneous sources with consistent naming/encoding.
- **Time-variant:** stores long historical horizons (years), unlike operational DBs (current values only).
- **Non-volatile:** no in-place updates — only initial load + read access (no transaction processing/recovery/concurrency-control needed, unlike OLTP).
- **ETL = Extract** (pull from heterogeneous sources) → **Transform** (clean, convert, aggregate) → **Load** (into the warehouse).
- **Architecture tiers:** Single-tier (rare) → Two-tier (adds a staging area before the warehouse) → **Three-tier (most common):** bottom = warehouse DB, middle = OLAP/application server (abstracted analysis view), top = user-facing tools.
- **Schemas:** **Star schema** (1 central fact table + surrounding dimension tables, denormalized, simplest/fastest). **Snowflake schema** (dimension tables further normalized into sub-tables — saves space, more joins needed). **Fact constellation / Galaxy schema** (multiple fact tables sharing dimension tables).
- **TRAP:** Snowflake = *normalized* dimensions (more joins, less redundancy); Star = *denormalized* dimensions (fewer joins, faster queries, some redundancy) — commonly reversed.

15.2 Data Cube & OLAP Operations

- **Data cube:** multidimensional view of data — **Dimension tables** (e.g., time, item, location) describe context; **Fact table** holds measures (e.g., dollars_sold) + foreign keys to dimensions.

```
Roll-up    : summarize / climb UP a concept hierarchy (or reduce a dimension)  -- e.g. city -> state -> country
Drill-down: opposite of roll-up; go to finer detail (more specific)             -- e.g. year -> quarter -> month
Slice      : fix ONE dimension to a single value (cube -> 2D)
Dice       : filter on TWO OR MORE dimensions (a sub-cube)
Pivot (rotate): reorient the cube axes for a different cross-tab view
```

- **TRAP:** Roll-up and Drill-down are exact opposites (aggregate-up vs detail-down); Slice (one dimension fixed) vs Dice (multiple dimensions filtered) is the pair most often confused.

15.3 Data Mining & KDD Process

- **Data Mining** = extracting **knowledge** (task-relevant information) from large data volumes; it is ONE step within the broader **KDD (Knowledge Discovery in Databases)** process.

```
KDD steps (in order): Data Cleaning -> Data Integration -> Data Selection ->
                        Data Transformation -> DATA MINING -> Pattern Evaluation -> Knowledge Presentation
```

- **Data preprocessing tasks:** Cleaning (fix missing/noisy/inconsistent data), Integration (merge sources), Transformation (normalize/aggregate), Reduction (smaller volume, same analytical value).
- **Major data mining tasks:** Classification & Prediction (assign records to predefined classes), Clustering (group by similarity — maximize intra-cluster, minimize inter-cluster similarity — NO predefined classes, unlike classification), Association Rule Mining (find frequent co-occurring itemsets), Outlier/Anomaly Analysis (find dissimilar objects), Evolution Analysis (trends over time).
- **TRAP:** Classification needs predefined class labels (supervised); Clustering discovers groups with no predefined labels (unsupervised) — the single most common mix-up in this topic.
- **Clustering approaches:** Partitioning (k-means, k-medoids — needs k specified upfront), Hierarchical (no k needed, builds a dendrogram), Density-based, Grid-based, Model-based.

15.4 Association Rule Mining — Apriori Algorithm

```
Support(X)      = (transactions containing X) / (total transactions)
Confidence(X->Y) = Support(X union Y) / Support(X)
Apriori (downward-closure) property: every subset of a frequent itemset must ALSO be frequent.
=> if a (k-1)-itemset is NOT frequent, no k-itemset containing it can be frequent (prune it).
Algorithm: L1 = frequent 1-itemsets
for k = 1, 2, ... while Lk is non-empty:
    Ck+1 = candidates generated by joining Lk with itself, then PRUNE using downward closure
    scan DB, count support of each candidate in Ck+1
    Lk+1 = candidates in Ck+1 meeting min_support
return union of all Lk
```

- A rule $X \rightarrow Y$ is output only if **both** $\text{Support}(X \cup Y) \geq \text{min_support}$ **and** $\text{Confidence}(X \rightarrow Y) \geq \text{min_confidence}$.

- **TRAP:** Apriori's efficiency comes entirely from the **downward-closure/anti-monotone property** — pruning candidates whose subsets aren't frequent avoids checking the full power set.
- Other frequent-pattern methods (name recognition only): **FP-Growth** (avoids candidate generation, uses a compact tree structure), **ECLAT** (vertical data format).

Worked Example

Itemset {Bread, PeanutButter} has Support = 60% (appears in 60% of transactions). If Support(Bread) = 80%, then Confidence(Bread → PeanutButter) = $60/80 = 75\%$.

15.5 Distributed Databases

- Sites are loosely coupled, share no physical hardware. **Homogeneous** (identical software/schema everywhere, cooperate fully, appears as one system to users) vs **Heterogeneous** (different schemas/software per site — harder query/transaction processing).
- **Replication:** multiple copies of data at different sites (faster reads, fault tolerance); **full replication** = every site has the entire DB.
- **Fragmentation:** splitting one relation across sites — **Horizontal** (row-wise split, each fragment has all columns, subset of rows) vs **Vertical** (column-wise split, each fragment needs the key attribute repeated to allow lossless rejoining).
- **Data transparency types:** Fragmentation transparency, Replication transparency, Location transparency — user need not know HOW/WHERE data is actually split/copied/stored.

15.6 NoSQL

- **"Not Only SQL"** — non-relational, schema-less/flexible-schema databases built for Big Data (Volume/Variety/Velocity) that rigid relational schemas handle poorly at scale.
- Trades strict **ACID** guarantees for **BASE**: **B**asically **A**vailable, **S**oft state, **E**ventual consistency.

CAP Theorem: a distributed system can guarantee at most 2 of the following 3 simultaneously:

- C - Consistency (every read gets the most recent write)
- A - Availability (every request gets a non-error response)
- P - Partition tolerance (system keeps working despite network partition/link failure)

- **TRAP: CAP theorem is THE defining fact of distributed/NoSQL systems** — since network partitions in a distributed system are essentially unavoidable (P is not optional), the real-world trade-off in practice is between **C and A** (e.g., choose CP or AP), not "pick any 2 freely."
- **NoSQL types** (4 categories):

Type	Structure	Examples
Key-Value	Hash table of key→value (JSON/BLOB/string)	Redis, DynamoDB, Riak, Memcached
Document	Key→value, but value is a structured document (JSON/XML)	MongoDB, CouchDB
Column-family	Data grouped/stored by column, not row (fast aggregates)	Cassandra, HBase
Graph	Nodes + edges storing entities and their relationships	Neo4j, OrientDB

- **TRAP:** Column-family stores are optimized for aggregate queries (SUM/COUNT/AVG) since a whole column is stored contiguously — the opposite strength profile of row-oriented RDBMS tables.

Questions

Q1. Which of the following is NOT one of the four defining properties of a data warehouse (per Inmon's definition)?

A. Subject-oriented B. Volatile C. Integrated D. Time-variant

Answer: B. Why: A data warehouse is explicitly **Non-volatile** (no in-place updates) — "Volatile" is the opposite of a defining property.

Q2. [Trap] A Snowflake schema differs from a Star schema in that:

A. Snowflake has no fact table B. Snowflake normalizes dimension tables into sub-tables, requiring more joins than the denormalized Star schema C. Star schema is always slower D. They are identical

Answer: B. Why: Snowflake schema trades query simplicity for reduced redundancy by further normalizing dimension hierarchies; Star schema keeps dimensions denormalized for simpler/faster queries.

Q3. Viewing sales data at the "country" level after previously viewing it at the "city" level is an example of:

A. Drill-down B. Roll-up C. Dice D. Pivot

Answer: B. Why: Moving from a more detailed level (city) to a more summarized level (country) is Roll-up (climbing the concept hierarchy); the reverse would be Drill-down.

Q4. [Trap] The key distinguishing feature between Classification and Clustering is:

A. Classification is unsupervised, Clustering is supervised B. Classification assigns records to predefined classes (supervised); Clustering groups by similarity with no predefined labels (unsupervised) C. They are the same technique D. Clustering requires labeled training data

Answer: B. Why: Classification learns from labeled examples to predict a known class; Clustering discovers natural groupings without any predefined class labels.

Q5. [Numerical] If Support({A,B}) = 40% and Support({A}) = 50%, what is Confidence(A → B)?

A. 40% B. 50% C. 80% D. 90%

Answer: C. Why: Confidence(A→B) = Support(AUB)/Support(A) = 40/50 = 80%.

Q6. The Apriori algorithm's efficiency relies fundamentally on which property?

A. Every superset of a frequent itemset is frequent B. Every subset of a frequent itemset must also be frequent (downward closure), letting infrequent candidates be pruned early C. Random sampling of transactions D. Sorting items alphabetically

Answer: B. **Why:** The anti-monotone/downward-closure property lets Apriori discard any candidate itemset that has an infrequent subset, without ever counting its actual support.

Q7. A relation is split by putting all rows for "North region" customers at Site 1 and "South region" customers at Site 2, with each fragment retaining all columns. This is:

A. Vertical fragmentation B. Horizontal fragmentation C. Full replication D. Star schema

Answer: B. **Why:** Splitting by ROWS while keeping all columns in each fragment is horizontal fragmentation; vertical fragmentation splits by columns instead.

Q8. [Trap] According to the CAP theorem, in the presence of an unavoidable network partition, a distributed NoSQL system must primarily trade off between:

A. Consistency and Partition tolerance B. Availability and Partition tolerance C. Consistency and Availability D. None of the above; all 3 are always achievable together

Answer: C. **Why:** Since partition tolerance is essentially mandatory for any real distributed system, the practical CAP trade-off in system design is between Consistency and Availability.

Q9. Which NoSQL database type is best suited for storing and querying entities along with their interconnections (e.g., social network friend relationships)?

A. Key-Value store B. Document store C. Column-family store D. Graph database

Answer: D. **Why:** Graph databases (e.g., Neo4j) natively store nodes and edges, making relationship-heavy queries (friend-of-a-friend, shortest path) efficient — unlike key-value or column stores.